GitHub Copilot Orchestration Framework

Version 1.3.1 — consolidated printable render, 2026-08-25
github.com/abhinavsehgal/github-copilot-orchestration-framework · MIT license
Generated from `docs/*.md` — the Markdown in the repo is the source of truth.

GitHub Copilot Orchestration Framework 1.3.1

00 — Quickstart: onboard any project (VS Code + Copilot CLI, many repos)

> The whole framework as one step-by-step walk. Every step says **what to do**, **why**, **what to paste**, and **how you know it worked**. Written for someone who has never seen the framework; the long version of each part is linked. Based on v1.2.0 and the GitHub Copilot + VS Code docs as verified on 2026-08-24 — if a step here disagrees with the chapter it links to, the chapter wins.
>
> **Time:** 15 min once per person · 2–4 h per repo · one afternoon for the workspace.

Part 0 · The words (read once)

Nine words. Everything else in this guide is made of these.

Word	Means
Orchestrator	The manager. Reads your task, decides who should do it, writes them a note, checks their work. **Never touches code itself.**
Specialist	A worker who does one kind of job (the API, the database, the UI, testing…). Touches only its own corner.
Handoff	The note the manager writes the worker: what to do, what the manager already knows (with proof), what would prove the manager wrong, what not to touch.
Return	The note the worker writes back: what it checked, what it changed, what it couldn't verify, what the manager must not pass on as fact.
Instruction file	A sticky note on a drawer. When anyone opens files in that drawer, the note appears. Lives in `.github/instructions/`.
Skill	A recipe card for a job you do again and again (`/commit-push-pr`, `/verify-build`). Lives in `.github/skills/`.
Hook	A tripwire. Runs a script when something happens (before a tool runs, when the agent tries to stop). Optional — add later.
Workspace	One desk with every repo's folder on it, plus one manager who only coordinates between repos. Its own small git repo.
Contract	The promise between a service and everyone who calls it (an API shape, an event payload, a shared package). The thing that breaks when two repos change at different times.

Part 1 · Before you start (15 minutes, once per person)

1. Make sure you have the four tools (plus Node.js only if you will turn on hooks)

Why: VS Code runs the agents; the Copilot CLI runs them from a terminal and is what the workspace uses to reach into other repos; git and `jq` are used by the workspace scripts. The shipped hook templates are Node scripts — any language works for a hook, these just happen to be `.mjs`.

```
# Each of these should print a version, not an error
code --version
copilot --version      # the GitHub Copilot CLI (the agentic one, not `gh copilot`)
git --version
jq --version
node --version         # only needed if you later turn on hooks
```

Missing `copilot`? Install it from GitHub's Copilot CLI page (docs.github.com → Copilot → Copilot CLI → Install). Missing `jq`? `brew install jq` on a Mac, `winget install jqlang.jq` on Windows.

✓ **You know it worked when:** the first four print a version (Node too if you plan on hooks), and in VS Code the Copilot Chat panel opens with **Agent** in the mode dropdown.

2. Get the framework onto your disk

Why: the framework is a folder of templates and two copy-paste prompts. You never install it into your project — you point Copilot at it.

```
mkdir -p ~/frameworks
git clone https://github.com/abhinavsehgal/github-copilot-orchestration-framework.git ~/frameworks/copilot
echo ~/frameworks/copilot     # remember this path — you will paste it into the prompts
```

✓ `ls ~/frameworks/copilot` shows `docs  prompts  templates`.

3. Read one file

Why: twenty minutes now saves a confused afternoon later. Skip the rest of the docs for now.

Open `docs/09-RUNBOOK.md`. That is the long version of Part 2 below.

✓ You can say in one sentence what the orchestrator does and does not do. (It coordinates. It never edits.)

Part 2 · One repo (2–4 hours — repeat for every repo)

Every repo gets its own manager and its own workers. Yes, every one — even a tiny service. The workspace in Part 3 only works if each repo already has this. Start with the repo you know best.

1. Open the repo in VS Code on a fresh branch

Why: the setup creates new files. A clean branch means you can throw it all away if you don't like it.

```
cd ~/code/<your-repo>
git checkout main && git pull
git checkout -b setup/copilot-orchestration
code .
```

✓ `git status` is clean and the bottom-left of VS Code says `setup/copilot-orchestration`.

2. Run the INVENTORY prompt (look, don't touch)

Why: Copilot scans the repo and *proposes* which specialists this repo needs. It writes nothing. You correct the proposal before anything is created.

Open Copilot Chat → set the mode dropdown to **Agent**. Open `~/frameworks/copilot/prompts/INVENTORY-PROMPT.md`, copy all of it, paste it into the chat. Replace `<framework path>` with the path from Part 1 step 2. Send.

It comes back with: a list of proposed specialists (usually 4–8), proposed instruction files with their folder globs, and a list of **Open Questions**. Answer the questions. Cross out specialists that feel wrong. Fewer is better.

> ⚠ **The orchestrator must be named** `<repo-name>-orchestrator` (for example `orders-orchestrator`). Not just `orchestrator`. In Part 3 every repo's manager sits on the same desk, and names must not collide.

✓ You have a short list of specialist names you agree with, and `git status` is still clean.

3. Run the BOOTSTRAP prompt (now it builds)

Why: this creates all the files. It first **backs up** anything already in `.github/` and asks before overwriting — so an existing `copilot-instructions.md` your team wrote is safe.

In the **same chat**, paste `~/frameworks/copilot/prompts/BOOTSTRAP-PROMPT.md` (path replaced again). It shows you each file before saving. Say yes to each one you agree with.

What it creates:

File	What it is
`.github/copilot-instructions.md`	The router. Short. Read by every Copilot request.
`.github/agents/<repo>-orchestrator.agent.md`	The manager.
`.github/agents/<name>.agent.md` ×N	The workers.
`.github/instructions/<area>.instructions.md`	The sticky notes, each with an `applyTo:` glob.
`.github/skills/<repo>-engineering/SKILL.md`	The six-gate working method every task follows.
`.github/skills/investigate-bug/`, `build-feature/`, `commit-push-pr/`, `correction-capture/`, `verify-build/` (`SKILL.md` each)	The recipe cards (`/investigate-bug`, `/build-feature`, `/commit-push-pr`, `/correction-capture`, `/verify-build`) — skills, so they also run on the cloud agent and CLI.
`docs/ai-context/PROJECT.md`	"What is true right now" — what's live where, verified commands.
`docs/ai-context/LEARNINGS.md`	Decisions, failed approaches, corrections.
`docs/ai-context/HANDOFF_SCHEMA.md`, `INDEX.md`, `GLOSSARY.md`	The note format, the map, the one-name-per-thing list.
`docs/<AREA>_BACKLOG.md`	Where "we'll do it later" must be written down.
`docs/ai-context/<area>.md`, `ORCHESTRATION_SPOONFEEDER.md`, `docs/_archive/README.md`	Per-area orientation skeletons, the human usage guide, the archive convention.

✓ `ls .github/agents` lists your orchestrator and specialists. Your normal build still passes.

4. Check the manager shows up

Why: an agent file with a typo in its name silently doesn't load.

In VS Code: **Developer: Reload Window** (Ctrl/Cmd+Shift+P). Open Copilot Chat. Click the agent dropdown (next to the mode dropdown).

✓ You see `<repo>-orchestrator` and every specialist in the dropdown.

> ⚠ Not there? The file must end in `.agent.md`, the `name:` line must match, and the YAML between the `---` lines must be valid. Fix, reload, look again.

5. Give it one real, small job

Why: the only proof the setup works is watching a handoff go out and a return come back.

Select `<repo>-orchestrator` in the dropdown. Type a real bug or tiny feature from your tracker — one that needs maybe 30 minutes of human work. Send.

Watch for three things: it restates the task; it writes a `handoff:` block (YAML) with `claims`, `failure_condition`, `in_scope`; the specialist answers with a `return:` block that lists `files_changed` and `tests_run`.

✓ You saw both blocks. The change is small and correct. If the specialist *refused* a vague handoff — that's the framework working, not failing.

6. Commit, open a PR, merge

Why: teammates get the agents the moment they pull. The backup folder stays local.

```
git add .github/ docs/ .gitignore
git commit -m "chore: bootstrap Copilot orchestration framework"
git push -u origin setup/copilot-orchestration
gh pr create --base main --title "Bootstrap Copilot orchestration"
```

✓ PR merged. A teammate pulls, reloads VS Code, and sees the same agents in their dropdown.

7. Later, not now: turn on hooks

Why: hooks are tripwires (block a stop while the build is red; force a correction to become an instruction file; refuse to end a turn after a production push with stale docs). Add them only after you've seen people skip the written rules. `docs/10-MECHANICAL-ENFORCEMENT.md` is the recipe.

```
# when you're ready:
mkdir -p .github/hooks .github/scripts
cp ~/frameworks/copilot/templates/hooks/hooks.json.template .github/hooks/framework.json
cp ~/frameworks/copilot/templates/hooks/*.mjs.template .github/scripts/
echo '.github/hooks/.state/' >> .gitignore      # the hooks' state file — never committed
# rename *.mjs.template → *.mjs, fill the <PLACEHOLDERS>, restart VS Code
```

> ⚠ Hooks are read when a session starts. After installing: reload VS Code / start a new `copilot` session, then test. Hook timeouts **fail open** — a slow check silently allows.

Now do Part 2 again for the next repo. Two repos done with working orchestrators is the minimum before Part 3.

Part 3 · The workspace (one afternoon, once per team)

Only when tasks keep saying "change the orders service *and* the web app". The workspace is a small extra repo that holds one coordinating manager, a map of who owns what, and the rules for changing contracts. It holds **no workers** — they stay in their repos. Long version: `docs/12-MULTI-REPO-WORKSPACES.md`.

1. Create an empty repo and clone it

Why: the workspace is its own repo so the map and the rules are versioned and shared. It has no CI and deploys nothing.

```
gh repo create <team>-workspace --private --clone
cd <team>-workspace
```

✓ You're inside an empty folder with a `.git`.

2. Copy the workspace templates into place

Why: fourteen files, each with a fixed home. The bootstrap script does the copying AND fills the per-repo placeholders, `.gitignore`, the `.code-workspace` folder list, the orchestrator's subagent allowlist and the service-map rows from `workspace.json` — so fill the manifest first (step 3) if you want the short path:

```
cp ~/frameworks/copilot/templates/workspace/workspace.json.template workspace.json   # fill it, then:
bash ~/frameworks/copilot/templates/workspace/bootstrap.sh.template ~/frameworks/copilot
```

Or by hand (`templates/workspace/README.md` has the same table):

```
T=~/frameworks/copilot/templates/workspace
mkdir -p .github/agents .github/instructions .github/skills/delegate scripts docs/ai-context
cp $T/copilot-instructions.md.template              .github/copilot-instructions.md
cp $T/workspace.code-workspace.template             <team>.code-workspace
cp $T/workspace.json.template                       workspace.json
cp $T/gitignore.template                            .gitignore
cp $T/orchestrator-agent.md.template                .github/agents/<team>-orchestrator.agent.md
cp $T/contract-guardian-agent.md.template           .github/agents/contract-guardian.agent.md
cp $T/service-mapper-agent.md.template              .github/agents/service-mapper.agent.md
cp $T/cross-repo-contracts.instructions.md.template .github/instructions/cross-repo-contracts.instructions.md
cp $T/delegate-skill/SKILL.md.template              .github/skills/delegate/SKILL.md
cp $T/sync-repos.sh.template scripts/sync-repos.sh
cp $T/delegate.sh.template   scripts/delegate.sh
cp $T/SERVICE_MAP.md.template docs/ai-context/SERVICE_MAP.md
cp $T/CONTRACTS.md.template   docs/ai-context/CONTRACTS.md
cp ~/frameworks/copilot/templates/HANDOFF_SCHEMA.md.template docs/ai-context/HANDOFF_SCHEMA.md
chmod +x scripts/*.sh
```

✓ `find . -type f -not -path './.git/*'` matches the layout in chapter 12.

3. Fill in the blanks – mostly in one file

Why: `workspace.json` is the source of truth: which repos, where they live, what each one's manager is called, and every contract between them. The scripts read it; you never hard-code a path anywhere else.

Open `workspace.json`. For each repo fill `name`, `path` (e.g. `services/orders`), `url`, `default_branch`, `orchestrator` (exactly the name from Part 2 – `orders-orchestrator`), `build`, `test`. For each contract fill `producer`, `consumers`, `spec` (where the API/event spec file lives), and a one-line `versioning` rule.

Then search every copied file for `<` and replace the remaining placeholders (`<WORKSPACE_SLUG>`, `<REPO_1_NAME>` …). The `.code-workspace` folder list must match the `path`s in `workspace.json`.

✓ `grep -rn "<[A-Z_0-9]*>" . --exclude-dir=.git` prints nothing. `jq . workspace.json` prints valid JSON.

4. Pull every repo onto the desk

Why: the children are plain clones in gitignored folders – disposable. The script clones what's missing and fetches what exists. It **never** switches or resets a branch; each repo's team owns that.

```
scripts/sync-repos.sh
```

✓ One `==` line per repo, no `⚠` warnings. A warning means that repo has no `<repo>-orchestrator` yet – go back to Part 2 for it.

5. Open the desk in VS Code

Why: VS Code reads each folder's `.github/` separately, so every repo's agents, sticky notes and skills are all available at once. The workspace file also switches on nested subagents.

File → Open Workspace from File… → pick `<team>.code-workspace`. Reload the window once it opens.

✓ The Explorer shows the workspace folder plus every child. The agent dropdown shows `<team>-orchestrator`, `contract-guardian`, `service-mapper` *and* each repo's own orchestrator.

> ⚠ If two repos both ship a worker called `backend-api`, on the desk those names collide. Rule: from the workspace, **only ever pick an orchestrator by name**. Never a worker. The workspace manager already knows this.

6. Commit the workspace

```
git add -A
git commit -m "chore: bootstrap cross-repo workspace"
git push -u origin main
```

✓ `git status` shows no child repo files – the clones are ignored, the map and rules are committed.

7. The first cross-repo job – add something

Why: the workspace's whole job is ordering changes across a contract: the service that *produces* a field changes first, is deployed, and only then do the apps that *read* it change.

Select `<team>-orchestrator`. Ask for something additive that crosses one contract, for example:

```
Add an `estimated_delivery` field to the orders API response and show it on the web order page.
```

Watch for: it reads `CONTRACTS.md`; it says "producer first: orders, then web"; it writes one handoff per repo, each with `repo:` and `contract_impact: additive`; it hands the orders job to `orders-orchestrator` (in chat) or runs `scripts/delegate.sh orders …` (from a terminal – that runs the orders repo's own Copilot session so its own rules and hooks apply).

✓ Two handoffs, in that order. Each return lists `consumers_grepped`. `git status` in the workspace root is clean – the workspace itself changed nothing.

8. Break it on purpose – rename something

Why: a rename is a breaking change. The correct answer is a refusal.

```
Rename `total` to `grand_total` in the orders API response.
```

✓ It refuses to run it as one change and proposes three: add `grand_total` → move every consumer → remove `total`. If it just did the rename, the contract rule is not loading – check `.github/instructions/cross-repo-contracts.instructions.md` has `applyTo: "**"`.

Part 4 · Every day (the whole thing in one table)

You want to…	Where	Do this
Fix a typo, ask a question	That repo, VS Code	Plain chat, no agent selected. The framework stays out of your way.
Do a medium/complex job inside one repo	That repo, VS Code	Pick `<repo>-orchestrator` in the dropdown. Describe the job. Read the handoff and the return.
Same, from a terminal	That repo, CLI	`cd` into the repo, run `copilot`, use `--agent=<repo>-orchestrator` or pick it inside. Same agents, same rules.
Ship what you did	Any	`/commit-push-pr` – it refuses to push to `main`, builds first, never stages secrets.
Change something in two or more repos	The workspace (`.code-workspace`)	Pick `<team>-orchestrator`. It orders the work producer → consumer and delegates one repo at a time.
Ask "is this API change safe?"	The workspace	Pick `contract-guardian`. It greps every consumer and answers with file:line, never an opinion.
Correct the agent ("no, use X")	Any	Then type `/correction-capture`. It turns the correction into a sticky note so it never happens again. Never accept "I'll remember".
Say "we'll do that later"	Any	It must be written into `docs/<AREA>_BACKLOG.md` before the chat ends. Spoken follow-ups vanish.
Push to production	That repo	Same session: update `CHANGELOG.md`, the affected `docs/ai-context/` map, and `PROJECT.md` "what is live where". A fresh agent sees only what's in git.
Hand a well-defined task to the cloud agent	GitHub issue	One issue per repo, assigned to that repo's orchestrator agent. It cannot touch two repos in one run.

Part 5 · When it goes wrong

You see	It usually means	Fix
The agent isn't in the dropdown	File name or frontmatter	Must end in `.agent.md`; `name:` must match; valid YAML between the `---` lines. Reload Window.
A sticky note never shows up	Its `applyTo:` glob doesn't match	Run `find . -path "<the glob>"` – if nothing prints, the glob is wrong. Globs are relative to that repo's root.
The wrong repo's worker answered	Name collision on the desk	From the workspace, invoke orchestrators only. The workspace router says so in rule 4.
`delegate.sh` exits with "handoff does not carry repo:"	The handoff file is missing its `repo:` line	Add `repo: <name>` exactly as in `workspace.json`. A note for one repo must never run in another.
`delegate.sh`: command not found / no such repo	`copilot` not on PATH, or a bad `path` in the manifest	`copilot --version`; then `jq '.repos[].path' workspace.json` and check each folder exists.
A hook never fires	Installed mid-session, or in the wrong place	Restart the session. Team hooks live in `.github/hooks/*.json` (committed); `~/.copilot/hooks` is only you. Timeouts fail open – keep checks fast.
The specialist refused the task	The handoff was vague	That's correct behaviour. Add the missing field it named (usually `failure_condition` or evidence on a claim) and re-send.
The manager is editing code itself	Specialists too narrow, or the wrong agent selected	Check the dropdown. If it really is the orchestrator, widen a specialist's scope rather than letting the manager code.
Something worked last month and now doesn't	The platform moved	Re-run `prompts/REFINEMENT-PROMPT.md`; its "platform drift" section re-checks the Copilot docs. Three framework claims went stale in three months once already (Pitfall 20).

Part 6 · Optional: let it run while you sleep (after two weeks)

Once real tasks have flowed through the framework for a couple of weeks, add your first **standing routine** – a narrow agent job on a schedule that opens small PRs behind review gates. The full pattern (seven conventions, starter catalog, budgets): `docs/13-STANDING-ROUTINES.md`.

- **What:** copy `templates/routine.md.template` to `docs/routines/<name>.md`, fill the charter, schedule it – a `schedule:` GitHub Actions workflow that assigns the routine's issue to the cloud agent, or runs headless `copilot -p --agent=<routine-agent>`.
- **Why:** maintenance becomes a stream instead of a project. The public reference fleet merged 180 routine PRs in a few weeks at ~1-in-50 noise.
- **Start with ONE of:** doc-drift checker · dead-code remover · flaky-test root-causer (in a workspace: the contract-drift checker – Part 3's guardian on a clock).
- **The safety rules:** PRs only · never self-merge · per-run budget + attempt caps · a checked completion write · one reporting channel.
- **You know it worked when:** every run posts to its channel – including "nothing to do" – and the first wrong PR produces a tuning-log entry in the charter, not just a closed PR.

The checklist (print this)

Per person, once - [ ] `code`, `copilot`, `git`, `jq` all print a version - [ ] Framework cloned to `~/frameworks/copilot`

Per repo - [ ] Branch `setup/copilot-orchestration` - [ ] INVENTORY prompt → specialist list agreed - [ ] BOOTSTRAP prompt → files created; orchestrator named `<repo>-orchestrator` - [ ] Reload Window → agents visible in the dropdown - [ ] One real small job → saw `handoff:` and `return:` - [ ] PR merged; a teammate sees the agents too - [ ] (later) hooks installed, session restarted, tested

Per team, once – after ≥ 2 repos are done - [ ] Workspace repo created; 14 template files in place; no `<PLACEHOLDER>` left - [ ] `workspace.json`: every repo + every contract filled - [ ] `scripts/sync-repos.sh` → no warnings - [ ] Opened the `.code-workspace`; workspace orchestrator + each repo's orchestrator visible - [ ] Additive cross-repo job → producer first, two handoffs, consumers grepped - [ ] Rename job → refused and split into add / move / remove

---

Cross-links

- https://abhinavsehgal.github.io/github-copilot-orchestration-framework/ (GitHub Pages) or `docs/00-QUICKSTART.html` offline – this guide plus the other two editions as one page with tabs (open in a browser; generated from the three `00-QUICKSTART.md` files on 2026-08-22).
- `docs/09-RUNBOOK.md` – the long version of Part 2.
- `docs/10-MECHANICAL-ENFORCEMENT.md` – hooks (Part 2 step 7).
- `docs/12-MULTI-REPO-WORKSPACES.md` – the long version of Part 3, with the verified platform behaviour behind every step.
- `docs/11-PROJECT-TRUTH-AND-LEARNINGS.md` – why the backlog, `PROJECT.md` and "push = freshen docs" rules exist.

- `docs/08-COMMON-PITFALLS.md` — Part 5, in full.
- `docs/13-STANDING-ROUTINES.md` + `templates/routine.md.template` — Part 6, in full (scheduled autonomy).

01 — Principles

The seven principles this framework rests on. Every other doc, prompt, and template implements these.

1. Orchestrator-worker pattern, documented not coded

One coordinator agent reads tasks, classifies them, picks the right specialists, and aggregates findings. Specialists do focused work in their own context (where the surface allows isolation).

The orchestration is **prescriptive documentation**, not runtime code. The "orchestrator" is a `.github/agents/<project>-orchestrator.agent.md` file with instructions. There is no orchestration service, no message queue, no central process.

This matters because: - It works with any Copilot surface that recognizes custom agents (Chat in VS Code / JetBrains / Visual Studio, the cloud agent on github.com, the Copilot CLI) - It's fully transparent — every rule is in version control as plain markdown - It's reversible — delete `.github/agents/`, `.github/instructions/`, `.github/skills/`, `.github/prompts/` (and `.github/hooks/` if installed) and you're back to default Copilot - Hard runtime enforcement (Copilot hooks — `.github/hooks/*.json`, v1.2) can be added later as a separate hardening pass

2. Each specialist runs with focused tool scope

Each specialist agent declares a `tools:` array in its frontmatter. Copilot enforces this at runtime — the agent physically cannot use tools not listed.

For specialists that need MCP server access (e.g. browser testing, database introspection), declare them in the `mcp-servers:` field with explicit scope.

Examples: - A `legal-compliance` specialist gets `tools: ['codebase', 'search', 'usages']` — no edit tools at all - A `frontend-ui` specialist gets edit tools plus a Chrome DevTools MCP for verification - A `backend-api` specialist gets edit tools plus a database MCP for schema introspection

This is the **single most important runtime defense** the framework provides. A REVIEW-ONLY specialist cannot hallucinate code into your repo because the harness blocks the tool call.

> **Note on context isolation:** Custom agents in Copilot Chat run within the active Chat session's context (unlike Claude Code subagents which run in fully isolated context windows). The cloud agent runs in true isolation. See `docs/06-INVOCATION-MODES.md` for the practical implications.

3. Universal evidence rule — "no evidence, no claim"

If a claim cannot be tied to a file:line, table:column, rule, doc anchor, test, or command output, it must be marked as an assumption (`confidence: low` outbound, or moved to `unverified_claims` inbound) and cannot be passed downstream as fact.

This rule appears verbatim in: - The handoff schema (both directions) - The orchestrator's "outbound discipline" - Every specialist's "incoming handoff validation" - Every path-globbed instruction file's preamble

It is the **core defense against cascading hallucinations** across hops. An orchestrator that hallucinates "the bug is in `foo.ts:42`" must cite that path; the specialist re-verifies before editing; if wrong, the specialist returns `status: claim_rejected` with the evidence of what it actually found.

4. Failure condition — articulate what would prove you wrong

Every outbound handoff includes a `failure_condition` field: one sentence stating what observable evidence would prove the orchestrator's hypothesis or delegation premise wrong. If the specialist observes that condition, it stops, returns `claim_rejected`, and includes the triggering evidence.

This is the **inverse of** `verify_before_acting`. Where `verify_before_acting` says "check these before acting," `failure_condition` says "if you observe this, STOP." Together they bracket the work in falsifiable claims.

Forcing the orchestrator to articulate `failure_condition` is itself a thinking discipline — if you can't name what would falsify your hypothesis, you don't have a hypothesis, you have a guess.

5. Tools are runtime-enforced; everything else is documentation discipline

Copilot's harness enforces: - `tools:` allowlists (specialist physically cannot use tools not listed in the agent file) - `disable-model-invocation:` (prevents auto-routing to this agent) - `user-invocable:` (controls manual selection) - `target:` (scopes agent to specific environments — `vscode` or `github-copilot`) - `mcp-servers:` per-agent MCP server scoping - The `applyTo:` glob in `.github/instructions/<NAME>.instructions.md` (auto-loaded only when matching files are touched) - Body length cap (30,000 chars)

Everything else is **documentation discipline** — the agent follows its instructions because they're in the system prompt: - Handoff schema fields being present - Refusal of vague delegations - Evidence rule being followed - `failure_condition` observation rule - Hop limits ("3rd same-specialist delegation → escalate") - "Read the right instruction file before editing" - Definition of Done completeness

Don't conflate the two layers. When you say "the orchestrator can only delegate to project specialists," that is runtime-enforced in VS Code (the orchestrator's `agents:` list is a subagent allowlist) and documentation on the cloud agent, which ignores the field (Pitfall 9). When you say "the legal-compliance agent cannot edit files," that IS runtime-enforced via the `tools:` field.

6. Three-tier documentation

Project documentation should split into three clearly-labeled tiers:

Tier	Location	Purpose	Read by
Orientation maps	`docs/ai-context/<topic>.md`	50-150 lines per area; gotchas; cross-links to canonical	Agents (per task routing)
Canonical references	`docs/<UPPERCASE>.md`	Architecture, API, schema, business – full detail	Humans + agents (when deeper)
Frozen archive	`docs/_archive/<date>/`	Sprint reports, post-mortems, migration logs, snapshots	Audits only – never linked from active docs

This separation makes drift impossible. Active docs live in the first two tiers. The archive is append-only and never referenced by agents/instructions/active docs.

The orientation maps live in `docs/ai-context/` even in a Copilot-only project – they're tool-agnostic, and using a tool-named directory (like `docs/copilot-context/`) would tie you to a single AI assistant.

7. Default Copilot stays default for inline completions

Do not put the orchestrator's persona in `.github/copilot-instructions.md`. The repository-wide instructions file is auto-loaded into EVERY Copilot interaction – including inline completions and code review. Loading the orchestrator's full persona there would: - Slow down inline completions - Pollute code-review prompts with delegation instructions - Force every interaction through the orchestrator's "delegate everything" body

`.github/copilot-instructions.md` should be a **thin router** – golden rules + workflow + cross-links. The orchestrator agent (`.github/agents/<project>-orchestrator.agent.md`) is invoked explicitly when the user picks it from the agent dropdown or types `@<project>-orchestrator` in chat.

Three invocation modes coexist: - **Default** – inline completions + plain Chat (uses repository-wide instructions only) - **Specialist mode** – Chat with a specific specialist agent selected - **Orchestrator mode** – Chat with `@<project>-orchestrator` for cross-domain work

See `docs/06-INVOCATION-MODES.md`.

What these principles get you

A team can work in different IDEs (VS Code / JetBrains / Visual Studio) and on github.com (cloud agent, code review) with consistent behavior. Specialists never inherit baggage from unrelated work. Orchestrator decisions are auditable (every claim cites evidence). Hallucinations don't compound across hops. New engineers see a clean `.github/` layout that tells them where to look.

The cost is upfront discipline: you spend a day writing the agent contracts, instruction files, and orientation maps. The payback is permanent – every future task benefits.

02 – Architecture

The physical layout of files in a project that uses this framework.

The full picture

```
your-project/
│
├── .github/
│   ├── copilot-instructions.md             ← repo-wide router (auto-loaded everywhere)
│   ├── agents/                             ← orchestrator + specialists
│   │   ├── <project>-orchestrator.agent.md
│   │   ├── <specialist-1>.agent.md
│   │   ├── <specialist-2>.agent.md
│   │   └── ...
│   ├── instructions/                       ← path-globbed invariants
│   │   ├── <domain-1>.instructions.md      (with applyTo: "src/api/**/*.ts")
│   │   ├── <domain-2>.instructions.md
│   │   └── ...
│   ├── skills/                             ← repeatable workflows, cross-surface (Agent Skills)
│   │   ├── <project>-engineering/SKILL.md  (invoked as /<project>-engineering, or auto-loaded)
│   │   ├── <workflow-1>/SKILL.md
│   │   └── ...
│   ├── prompts/                            ← repeatable prompts, IDE-only (slash commands)
│   │   ├── <workflow-1>.prompt.md          (invoked as /<workflow-1>)
│   │   └── ...
│   ├── hooks/                              ← optional: lifecycle hooks (Chapter 10)
│   │   └── <name>.json                     (preToolUse / postToolUse / agentStop …)
│   └── chatmodes/                          ← RETIRED — rename *.chatmode.md → agents/*.agent.md
│
├── AGENTS.md                                ← optional cross-AI metadata file
│                                              (also recognized by other AI tools)
│
├── docs/
│   ├── ai-context/                          ← orientation maps (read by agents)
│   │   ├── INDEX.md
│   │   ├── PROJECT.md                       ← current truth: what is live where (Chapter 11)
│   │   ├── LEARNINGS.md                     ← decisions, failures, corrections (Chapter 11)
│   │   ├── GLOSSARY.md                      ← one name per concept (Chapter 11)
│   │   ├── HANDOFF_SCHEMA.md
│   │   ├── ORCHESTRATION_SPOONFEEDER.md
│   │   └── <area>-experience.md             ← per-domain orientation, 50–150 lines each
│   ├── ARCHITECTURE.md                      ← canonical refs (full detail)
│   ├── API.md
│   ├── <YOUR-OTHER-CANONICAL-DOCS>.md
│   ├── <AREA>_BACKLOG.md                    ← deferred work, one per area (Chapter 11)
│   └── _archive/                            ← frozen historical
│       ├── README.md
│       └── <YYYY-MM>/
│
└── <your application code>/
```

What lives where, and why

`.github/copilot-instructions.md` (root router)

Auto-loaded by Copilot for EVERY interaction in this repository — including inline completions, Chat, code review, and cloud agent. Should be **under 200 lines**.

Contains: - Identity (one sentence about the project) - Golden rules (security, deployment, branching) - Mandatory workflow (numbered steps for every non-trivial task) - Routing guidance (“for X tasks, use the @ agent”) - Cross-links to the spoonfeeder, INDEX, and canonical docs

Does NOT contain: - Long technical detail (lives in canonical docs) - Per-domain gotchas (lives in `.github/instructions/`) - The orchestrator’s full persona (lives in `.github/agents/<project>-orchestrator.agent.md`) - Sprint history or audit content (lives in `docs/_archive/`)

⚠ **Size budget.** The current docs say repository instructions “must be no longer than 2 pages” (and any single instruction file should stay under about 1,000 lines). Front-load the rules that matter most for code review — good practice, not a hard cap; the older “code review reads only the first 4,000 characters” sentence is no longer on the docs (Pitfall 2).

`.github/agents/`

One markdown file per agent, named `<agent-name>.agent.md` (a bare `.md` name still works; `.agent.md` is the current convention). Frontmatter declares fields per the Copilot custom agents spec:

```
---
name: <agent-name>                    # optional, defaults to filename
description: <required string>        # how the agent decides when to delegate
tools: ['tool-a', 'tool-b']           # optional allowlist
mcp-servers: { ... }                  # optional per-agent MCP scoping
model: <model-name>                   # optional, IDE-specific
target: vscode | github-copilot       # optional environment scoping
disable-model-invocation: true|false  # optional, prevents auto-routing
user-invocable: true|false            # optional, controls manual selection
metadata: { key: value }              # optional annotations
agents: ['<specialist-1>', ...]       # VS Code only: subagent allowlist (`*` = all); ignored by the cloud agent
---
```

VS Code additionally documents `handoffs`, `argument-hint` and `hooks` (preview, agent-scoped); `argument-hint` and `handoffs` are not supported on the cloud agent. User-level agents live in `~/.copilot/agents/`, and VS Code also reads `.claude/agents/` by default (`chat.agentFilesLocations`).

Body is the agent's system prompt — up to 30,000 characters.

Always includes exactly one orchestrator (`<project-slug>-orchestrator`). Specialists are added based on the project's natural domain boundaries (typically 5-12 specialists for a medium-complexity codebase).

See `03-AGENTS-GUIDE.md` for how to design agents.

`.github/instructions/`

Path-globbed invariants — Copilot's direct equivalent of "rules" in other frameworks. Each file:

```
---
applyTo: "src/api/**/*.ts,src/server/**/*.ts"
excludeAgent: "code-review"          # optional: don't apply during code review
---

# API Layer Rules

## Hard rules

### <Rule title>
**Why:** <reason>
**How to apply:** <how>
```

Frontmatter `applyTo:` accepts glob patterns (comma-separated for multiples). When Copilot is editing a matching file, this instruction file is auto-loaded into the relevant prompts.

These are the **hard-won gotchas** of your codebase — things that broke production once and must not break again.

`.github/skills/`

Agent Skills — the **cross-surface** home for repeatable multi-step workflows, and the Copilot equivalent of Claude Code skills. One directory per skill, `SKILL.md` inside:

```
---
name: <skill-name>                    # required; lowercase-hyphen; equals the directory name
description: <one sentence — when to use this>   # required
---

<the procedure — steps, checks, Definition of Done>
```

Invoked as `/<skill-name>`, or loaded automatically when the task is relevant. Supported by the cloud agent, code review, the Copilot CLI, and agent mode in VS Code and JetBrains. Also discovered from `.claude/skills/` and `.agents/skills/`; personal skills live in `~/.copilot/skills/`. See Chapter 5 for the full frontmatter.

Use skills for bug investigation, feature build, QA pass, compliance review, and the project's engineering playbook (`templates/engineering-playbook-skill.md.template`).

`.github/prompts/`

IDE-only repeatable prompts invoked via slash command. Each file:

```
---
description: <one sentence — appears in /command picker>
---

<the prompt body — instructions, may include ${input:variable:hint} placeholders>
```

In Copilot Chat, type `/<filename-without-extension>` to invoke. Example: `.github/prompts/investigate-bug.prompt.md` is invoked as `/investigate-bug`.

Available in: VS Code, Visual Studio, JetBrains only. Currently in **public preview**. The cloud agent and the Copilot CLI do not use prompt files — VS Code's advice for a prompt an agent must run is "convert it to an agent skill". A prompt file is a convenience for a workflow you only ever start by hand in the IDE; anything else belongs in `.github/skills/`.

`.github/hooks/` (optional)

Lifecycle hooks — `<name>.json` files declaring commands to run on `sessionStart`, `userPromptSubmitted`, `preToolUse`, `postToolUse`, `agentStop` and other events. The cloud agent loads only `.github/hooks/*.json`; the CLI also reads `~/.copilot/hooks/`; VS Code also reads `.claude/settings.json`. This is where the framework's mechanical enforcement (build-gate, doc-freshness gate, correction-capture) lives — see `docs/10-MECHANICAL-ENFORCEMENT.md` and `templates/hooks/`.

`.github/chatmodes/` — RETIRED

Custom chat modes (`*.chatmode.md`) were the earlier name for custom agents and are retired. Rename any existing file to `.github/agents/<name>.agent.md`; the `description`, `tools` and `model` fields carry over unchanged (Pitfall 7). Do not create new chat-mode files.

`AGENTS.md` (optional, recommended for cross-AI projects)

A cross-AI standard file recognized by GitHub Copilot, Claude, Gemini, and other AI assistants. If your project uses multiple AI tools, put shared cross-AI metadata here.

Format: plain markdown describing the project, agents, and conventions.

`docs/ai-context/`

Orientation maps. Per-area guides Copilot reads at the start of any task in that area. Cheaper than reading thousand-line canonical docs.

Format: 50-150 lines per file.

Audience: AI agents (primary), humans (secondary).

Naming: `<area>-experience.md` or `<area>-<concern>.md`.

Special files: - `INDEX.md` — task-type → docs + agent map - `HANDOFF_SCHEMA.md` — bidirectional handoff schema - `ORCHESTRATION_SPOONFEEDER.md` — human-facing usage guide

The `docs/ai-context/` location is tool-agnostic so the same orientation maps work if you later add Claude Code, Gemini Code Assist, or other AI tools.

`docs/<UPPERCASE>.md`

Canonical references. Full detail. Updated when the underlying truth changes. As long as needed (no artificial line cap).

Common files: `ARCHITECTURE.md`, `API.md`, `TECH_STACK.md`, `CHANGELOG.md`, `PRODUCT_REQUIREMENTS.md`, `BUSINESS_DOCUMENT.md`.

`docs/_archive/`

Frozen historical material. Date-prefixed subdirectories (`<YYYY-MM>/`). Active docs never link here.

Required: `_archive/README.md` documenting the archive convention.

Why this layout works

Predictability for Copilot

Every agent knows where to find: - Its own contract: `.github/agents/<self>.agent.md` - Repository-wide rules: `.github/copilot-instructions.md` (auto-loaded) - Path-specific rules: `.github/instructions/*.instructions.md` (auto-loaded when editing matching files) - Reusable workflows: `.github/skills/*/SKILL.md` (invoked via `/<name>` on every surface) — plus IDE-only `.github/prompts/*.prompt.md` - Mechanical enforcement: `.github/hooks/*.json` (if installed) - Current truth and learnings: `docs/ai-context/PROJECT.md`, `docs/ai-context/LEARNINGS.md` - The handoff schema: `docs/ai-context/HANDOFF_SCHEMA.md` - Orientation per topic: `docs/ai-context/<area>.md` - Canonical references: `docs/<UPPERCASE>.md`

Predictability for humans

A new engineer joining the project sees: - `.github/copilot-instructions.md` — tells them how Copilot is configured - `.github/agents/`, `.github/instructions/`, `.github/skills/`, `.github/prompts/`, `.github/hooks/` — Copilot config files - `docs/` — documentation - Application code in its standard tech-stack location

They can ignore the `.github/` agents/instructions/skills/prompts/hooks if they don't use Copilot.

Minimal coupling to tech stack

Nothing in `.github/` or `docs/ai-context/` cares about your framework, language, or deploy target. The agents reference paths in your actual codebase, but the agent FILES themselves are pure markdown. You can reorganize your codebase entirely and only need to update the `applyTo:` globs and the orientation maps' cross-links.

What does NOT belong in this framework

- **Application code.** Stays where your tech stack puts it.
- **Build config.** Stays at root.
- **CI workflows.** Stays in `.github/workflows/`.
- **Test files.** Stays in your tests directory.
- **Generated artifacts.** Gitignored.

The framework is **purely additive** to whatever else lives in your repo.

Variations

Monorepo

For monorepos, you can either: - Have one `.github/` at the root that knows about all packages, or - Have one `AGENTS.md` per package + a root `.github/copilot-instructions.md` that orchestrates across packages

The first is simpler. The second is more isolated. Pick based on whether tasks routinely cross package boundaries. Note that `AGENTS.md` can exist at any depth and the nearest one in the directory tree wins (cloud agent, code review, CLI); VS Code reads the root one automatically and nested ones only behind the experimental `chat.useNestedAgentsMdFiles` setting.

Multiple repositories (web + mobile + microservices)

When the units are separate repos rather than packages, see `12-MULTI-REPO-WORKSPACES.md` — the cloud agent cannot change more than one repository in a run, so cross-repo work is a workspace of per-repo installs plus shared contracts, not one agent over everything.

Mobile + web split

If your project has separate mobile and web codebases in one repo, your orchestrator typically delegates to a `mobile-ui` specialist OR a `web-ui` specialist. They can share a `backend-api` specialist for the API-side work.

Multi-tenancy / multi-product

If you serve multiple products from one codebase, consider per-product orientation maps and per-product instructions. The orchestrator can be one or per-product depending on whether tasks routinely cross products.

Heavy infra / DevOps focus

Add specialists for `infrastructure`, `observability`, `release-automation`. Their instruction files cover Terraform/CDK/Kubernetes/Helm conventions.

ML / data heavy

Add specialists for `data-pipeline`, `model-training`, `inference-serving`. Their orientation maps cover dataset locations, experiment tracking, model versioning.

The architecture flexes; the principles don't.

Organization-level customization (Business / Enterprise)

If you have GitHub Copilot Business or Enterprise, you can also configure:

- **Organization custom instructions** (UI at organization settings) — applies to Chat on github.com, code review, and cloud agent across all org members
- **Organization-level custom agents** — `/agents/<NAME>.agent.md` in the organization's `.github` or `.github-private` repository

Use organization-level configuration for company-wide policies (e.g. "always cite the file:line for security-sensitive code"). Use repository-level for project-specific specialists and rules.

Precedence (highest to lowest): Personal > Repository > Organization. All sets are provided to Copilot when relevant.

03 – Agents Guide (GitHub Copilot)

How to design the orchestrator and specialists for your project using GitHub Copilot's custom agents feature.

The orchestrator

Every project has exactly **one** orchestrator. Naming convention: `<project-name>-orchestrator` (e.g. `acme-orchestrator`, `payments-platform-orchestrator`).

Responsibilities

- Read incoming task. Restate it.
- Identify affected roles / domains.
- Read minimum docs (`docs/ai-context/INDEX.md` → 1-3 orientation maps).
- Pick the right specialists (typically 1-3 per task).
- Issue structured handoffs with claims + evidence + failure_condition.
- Receive returns. Verify rejected_claims. Re-verify unverified_claims before propagating.
- Aggregate findings into a final summary with the project's "Definition of Done."

Constraints

- **Restricted tool set** – primarily Read/Search tools, NOT edit tools. The orchestrator coordinates; specialists implement.
- `disable-model-invocation: false` – auto-routable when the task is complex.
- `user-invocable: true` – explicitly selectable from agent dropdown.

Frontmatter template

```yaml
---
name: <project>-orchestrator
description: Main task dispatcher for <Project>. Use for any medium/complex task. Picks the minimum docs and specialists; never loads
      everything. Reference other agents using the agent tool alias when delegating.
tools: ['codebase', 'search', 'usages', 'problems', 'changes']
agents: ['<specialist-1>', '<specialist-2>', 'qa-functional', 'security-privacy']   # VS Code subagent allowlist; ignored by the cloud agent
target: github-copilot
user-invocable: true
disable-model-invocation: false
---
```

File name: `.github/agents/<project>-orchestrator.agent.md`. (Field reference: see Custom agents configuration – exact tool names depend on Copilot extension version. `agents:` is documented by VS Code only – it names the agents this one may invoke as subagents, `*` meaning all; the cloud agent ignores the field, so keep the same list in the body too.)

Specialists

A specialist owns one domain. Specialists are domain-shaped, not technology-shaped.

Bad: `react-specialist`, `typescript-specialist`. Good: `frontend-ui`, `payments`, `realtime-sync`, `compliance`.

How many do you need?

Project size	Specialist count
Solo project / prototype	0-2 (just use plain Copilot Chat)
Small team / single product	4-6
Medium codebase / multiple roles	6-10
Large codebase / multi-product	10-15

More than 15 is a smell – you're probably making them too narrow.

Common specialists by project type

Web app (any stack)

Specialist	Domain
`frontend-ui`	UI components, routing, state management, client-side interactions
`backend-api`	API routes, server logic, request handling
`database`	Schema, migrations, query layer, indexes
`auth-security`	Authentication, authorization, sessions, secrets
`qa-functional`	Browser/E2E testing, regression checks
`release-devops`	Build, deploy, env config, observability

Mobile app

Specialist	Domain
`mobile-ui`	Screens, navigation, gestures, platform-specific UX
`mobile-state`	State management, persistence, offline sync
`mobile-native`	Platform bridges (iOS/Android native or Flutter plugins)
`backend-api`	Server-side endpoints
`qa-mobile`	Device testing, simulator/emulator flows

Backend / API service

Specialist	Domain
`api-routes`	HTTP/gRPC route handlers
`domain-logic`	Core business logic
`data-layer`	DB access, repositories, ORM patterns
`integrations`	Third-party API clients
`observability`	Logging, metrics, tracing
`release-devops`	Deploy, env, secrets

Cross-cutting (add to most projects)

Specialist	Domain
`product-flow`	Acceptance criteria, cross-role behavior (LIGHT EDITS ONLY)
`qa-functional`	Tests across the system
`security-privacy`	Auth/sensitive-data review (REVIEW-ONLY)
`legal-compliance`	Regulatory review (REVIEW-ONLY)
`release-devops`	Build/deploy/cron/env
`context-librarian`	Maintains `.github/`, `docs/ai-context/`, archives drift

REVIEW-ONLY specialists

Some specialists should explicitly **not** edit code. Tag them in the description, give them only read tools (no `edit_files` / `create_file` / `apply_patch`), and document the constraint in their I CANNOT section.

Examples: - `legal-compliance` — flags risks; never claims final legal advice - `security-privacy` — identifies vulnerabilities; doesn't ship fixes - `architecture-review` — reviews proposed designs; doesn't implement

Frontmatter for REVIEW-ONLY:

```
---
name: legal-compliance
description: REVIEW-ONLY. Flags <list of regulations> risks. Produces attorney-checklist content. Never claims final legal advice.
tools: ['codebase', 'search', 'usages', 'problems', 'changes', 'fetch']
target: github-copilot
user-invocable: true
disable-model-invocation: false
---
```

Notably absent: edit-file tools, terminal/run tools. The harness physically prevents this agent from modifying files.

⚠ **Verify your IDE's exact tool names.** The `tools:` array names are tool identifiers Copilot recognizes – they vary slightly between extension versions and IDEs. The list above (`codebase` , `search` , `usages` , `problems` , `changes` , `fetch`) is a common read-only set. Check your IDE's agent configuration UI to see the current valid identifiers, or consult the Copilot documentation.

Implementation specialists

Get edit/write/terminal access plus relevant MCP server scope.

Frontmatter for an implementation specialist:

```
---
name: backend-api
description: Builds and fixes API routes, server logic, request handling. Coordinates with database and auth-security specialists.
tools: ['codebase', 'search', 'usages', 'problems', 'changes', 'edit_files', 'apply_patch', 'runCommands']
target: github-copilot
user-invocable: true
disable-model-invocation: false
---
```

For specialists that need MCP server access (e.g. browser testing, database introspection), declare via `mcp-servers:` :

```
---
name: qa-functional
description: Browser-based E2E testing and regression checks across roles.
tools: ['codebase', 'search', 'usages', 'changes', 'edit_files', 'apply_patch', 'runCommands']
mcp-servers:
  playwright:
    type: 'local'
    command: 'npx'
    args: ['-y', '@modelcontextprotocol/server-playwright']
    tools: ['*']
---
```

(Exact MCP server config syntax may vary – check the Copilot MCP documentation.)

Agent body structure (every agent)

Every agent file body should include these sections in order:

```
# <Agent Display Name>

<2-sentence description of the agent's domain.>

## When to use

- Bullet list of task types this agent owns

## When NOT to use

- Bullet list of task types that belong to other agents

## Required reading

1. `docs/ai-context/INDEX.md` — task → docs map
2. <area-specific orientation map>
3. **`.github/instructions/<your-instructions>.instructions.md`** is auto-loaded when editing matching paths
4. <other rules / canonical refs>

## Incoming handoff validation

<paste the standard incoming handoff validation block — see template>

## Return schema (required)

<paste the standard return schema block — see template>

## I CAN

- Bullet list of what this agent is allowed to do

## I CANNOT

- Bullet list of what this agent must refuse
- Always include: "Push to main" (or your equivalent protected branch)

## Definition of Done

1. Files changed (paths)
2. Instruction files referenced before editing
3. Tests run
4. Risks remaining
5. (any agent-specific Definition of Done items)

## Cross-links

- `<related instruction file>`
- `<related orientation map>`
- `<canonical doc>`
```

The "Incoming handoff validation" and "Return schema" sections are **identical across all specialists** — they enforce the universal handoff contract. See `templates/specialist-agent.md.template`.

Cross-agent invocation

In Copilot, one agent can invoke another. On the cloud agent this is the `agent` tool alias, which "allows a different custom agent to be invoked to accomplish a task"; in VS Code it is the `runSubagent` tool, gated by the invoking agent's `agents:` frontmatter list:

```
@<other-specialist-name> <task-with-handoff-yaml-block>
```

Whether there is an **allowlist** depends on the surface (verified 2026-08-22):

- **VS Code:** `agents: ['<specialist-1>', …]` on the orchestrator is a real allowlist — an agent not named cannot be invoked as a subagent (`*` = all; never use `*` on an orchestrator). A subagent invoking its own subagents is off unless `chat.subagents.allowInvocationsFromSubagents` is enabled. This is the equivalent of Claude Code's `Agent(specialist-1, specialist-2)` allowlist.
- **Cloud agent (github.com):** no per-agent allowlist is documented. The `agents:` field is ignored; the orchestrator's body is the only routing table, and respecting it is documentation discipline.

Either way, declare `agents:` on the orchestrator and repeat the list in its body — one line gives you enforcement on one surface and a machine-readable contract on both.

Specialists do not chain. Even where the platform allows a specialist to invoke another specialist, by framework convention a specialist returns to the orchestrator with `recommended_next_agent` and lets the orchestrator issue the next handoff. A specialist that invokes another has become an un-audited orchestrator: the handoff schema, the evidence rule and the orchestrator's return-validation are all bypassed one level down. Leave `agents:` off specialists entirely (or empty). The only sanctioned nesting is orchestrator → orchestrator (Chapter 12, multi-repo), where both hops carry the full schema.

Naming conventions

- Lowercase with hyphens: `backend-api`, `frontend-ui`, `legal-compliance`

- Filename matches `name`, with the `.agent.md` suffix: `backend-api` → `backend-api.agent.md` (a bare `backend-api.md` still loads; `.agent.md` is the current convention and is what a renamed `.chatmode.md` becomes — Pitfall 7)
- Be specific but not over-narrow: `payments` good, `stripe-webhook-handler` too narrow
- Cross-cutting concerns get their own agent: `qa-functional`, `security-privacy`, not folded into another specialist

When to add a new specialist vs. extend an existing one

Add a new specialist when: - The domain has its own gotchas (would warrant its own instruction file) - Tasks in that domain frequently arrive - The existing specialists' "I CANNOT" sections would all reject this work

Extend an existing specialist when: - The work overlaps with an existing specialist's domain - The new domain is small (fits in 1-2 paragraphs of the existing agent's body) - You haven't seen 3+ tasks in this domain yet

Anti-patterns

❌ **Technology-named specialists.** `typescript-specialist` doesn't have a clear domain.

❌ **Layer-named specialists.** `database-specialist` is fine if "database" means a domain (schema, migrations); not fine if it means "anyone touching SQL anywhere should consult me."

❌ **Specialists with overlapping `tools` and overlapping descriptions.** Confuses the auto-routing logic.

❌ **Specialists with no `tools:` field.** They inherit ALL tools, defeating defense-in-depth. Always declare `tools` explicitly.

❌ **Specialists that invoke other specialists.** VS Code allows it (via `agents:`) and the cloud agent's `agent` tool does not stop it, but a specialist that does so has become an un-audited orchestrator — the handoff schema, the evidence rule and the orchestrator's return-validation are all bypassed one level down. By framework convention a specialist returns `recommended_next_agent` and lets the orchestrator chain the work (see "Cross-agent invocation").

❌ **Specialists whose "I CAN" includes "approve production deploys."** That's the project owner's call.

❌ **Putting the orchestrator's persona in `.github/copilot-instructions.md`.** That file is loaded for inline completions and code review too — pollutes everything. Keep the orchestrator persona in `.github/agents/<project>-orchestrator.agent.md`, separate.

A note on cloud agent vs IDE chat agents

The same `.github/agents/<NAME>.agent.md` file works for both: - **IDE Chat custom agents** (VS Code, JetBrains, etc.) - **Cloud agent on github.com** (autonomous agent that takes issue/PR assignments)

Some properties may behave differently between environments: - The cloud agent runs in true context isolation - The IDE chat agent runs within the active Chat session's context - The `target:` field can scope an agent to one or the other (`vscode` vs `github-copilot`) - Some properties are VS Code-only (`agents`, `handoffs`, `argument-hint`, `hooks`): `argument-hint` and `handoffs` are explicitly not supported on the cloud agent per the VS Code docs, and `agents:` is ignored there (Pitfall 9)

For most projects, write agents that work in BOTH environments by avoiding IDE-specific properties. Use `target:` only when an agent genuinely needs to be exclusive to one environment.

04 — Handoff Schema (GitHub Copilot)

The bidirectional schema for orchestrator ↔ specialist communication. This is the framework's central artifact.

The schema is **identical** to the equivalent in the Claude Code framework — it's tool-agnostic. What changes is how it's invoked: in Copilot, the orchestrator emits the YAML block when it `@`-mentions a specialist or when the cloud agent is delegating work.

Why a schema

Without structure, handoffs are free-form prompts. Specialists guess what's expected. Orchestrators don't commit to specific claims. Cascading hallucinations compound across hops because nothing forces evidence-to-claim binding at the boundary.

The schema closes both gaps with two YAML blocks (one per direction) and one universal evidence rule.

Universal evidence rule

> **No evidence, no claim.** If a claim cannot be tied to a file, line, table, column, rule, doc, test, or command output, it must be marked as an assumption (`confidence: low` outbound, or moved to `unverified_claims` inbound) and cannot be passed downstream as fact.

This rule applies to both directions of every handoff. It is repeated verbatim in the orchestrator's outbound-discipline section and in every specialist's incoming-handoff-validation section.

Outbound: orchestrator → specialist

When the orchestrator delegates to a specialist (via `@<specialist-name>` in Chat or by spawning the specialist in the cloud agent flow), the prompt body MUST include this YAML block at the top:

```yaml
handoff:
  schema_version: 1
  handoff_id: <short unique id, e.g. h-2026-05-02-a3f>
  from_agent: <orchestrator-name>
  to_agent: <specialist-name>
  goal: <one sentence — what done looks like>
  task_class: <bug | feature | review | qa | refactor | audit>
  affected_roles: [<role1>, <role2>, ...]
  claims:
    - claim: <factual statement the orchestrator commits to>
      evidence: <path/to/file.ts:42 OR instruction:.github/instructions/foo.instructions.md OR doc:docs/ARCHITECTURE.md#section OR
        table:column OR test:path OR command:output>
      confidence: <high | medium | low>
  verify_before_acting:
    - <claim or assumption the specialist MUST re-verify against source before editing>
  failure_condition: <one sentence — observable evidence that proves the orchestrator's hypothesis or delegation premise is wrong. If the
      specialist observes this, it MUST stop and return status: claim_rejected (or needs_clarification) instead of editing on a false
      premise.>
  in_scope:
    - <path or component the specialist may edit>
  out_of_scope:
    - <path or component the specialist must NOT touch — even if "while we're in there">
  instructions_to_read:
    - <.github/instructions/<NAME>.instructions.md path that applies to in-scope files, OR "specialist will discover from applyTo: globs">
  expected_artifacts:
    - <file changed, test added, summary section, etc.>
  hop_context:
    parent_task: <one line — why the orchestrator is delegating>
    previous_specialist: <agent name or "none">
    previous_findings_summary: <one line — what came back, or "n/a">
```

Field requirements (outbound)

Field	Required	Notes
`schema_version`	yes	Currently `1`. Bump on breaking changes.
`handoff_id`	yes	Short unique id. Specialist echoes it on return so they pair.
`from_agent`	yes	Always the orchestrator's name.
`to_agent`	yes	The specialist's `name` field from its `.github/agents/<NAME>.agent.md` frontmatter.
`goal`	yes	One sentence with measurable outcome. "Fix" or "improve" without an outcome is too vague.
`task_class`	yes	One of the six values.
`affected_roles`	yes	Non-empty list.
`claims`	yes (may be empty list)	Every entry MUST have `evidence`. Use `confidence: low` for assumptions.
`verify_before_acting`	yes if task touches code	Empty allowed only for pure-research handoffs.
`failure_condition`	yes	One sentence. The inverse of `verify_before_acting`.
`in_scope`	yes	Non-empty list.
`out_of_scope`	yes	Empty list `[]` is valid. Missing key is not.
`instructions_to_read`	yes	Use `["specialist will discover from applyTo globs"]` if you don't know which instructions apply.
`expected_artifacts`	yes	Drives the specialist's `files_changed` / `tests_run` later.
`hop_context`	yes	All three sub-fields required. Use `none` / `n/a` for first hop.

Inbound: specialist → orchestrator (return schema)

Every specialist response MUST end with this YAML block:

```yaml
return:
  schema_version: 1
  handoff_id: <copy from inbound — pairs the response>
  from_agent: <specialist-name>
  to_agent: <orchestrator-name>
  status: <completed | blocked | needs_clarification | claim_rejected>
  verified_claims:
    - claim: <orchestrator claim that was confirmed against source>
      evidence: <path:line or command output the specialist saw>
  rejected_claims:
    - claim: <orchestrator claim that turned out to be wrong>
      evidence: <what the specialist actually found>
  unverified_claims:
    - claim: <orchestrator claim that was not checked>
      reason: <why — out of scope, infra not available, etc.>
  evidence_used:
    - <path:line, instruction file, doc anchor, or command output>
  files_changed:
    - <path:line range and one-line summary>
  tests_run:
    - <command + result>
  risks:
    - <risk + mitigation or "none">
  recommended_next_agent:
    agent: <specialist-name or "none">
    why: <one line>
  do_not_pass_downstream_without_verification:
    - <claim or finding the orchestrator must NOT propagate as fact>
```

Field requirements (inbound)

Field	Required	Notes
`schema_version`	yes	Must match outbound.
`handoff_id`	yes	Must echo inbound id verbatim.
`from_agent` / `to_agent`	yes	Self-explanatory.
`status`	yes	`completed` only if all `verify_before_acting` items checked. Use `claim_rejected` when an orchestrator claim was wrong.
`verified_claims`	yes (may be `[]`)	Empty list is valid. Missing key is not.
`rejected_claims`	yes (may be `[]`)	Empty list is valid. Non-empty implies `status: claim_rejected` or `blocked`.
`unverified_claims`	yes (may be `[]`)	Anything you accepted without re-checking goes here.
`evidence_used`	yes	Real paths/commands. "I read the codebase" is not evidence.
`files_changed`	yes (may be `[]`)	Empty list valid for review/audit handoffs.
`tests_run`	yes (may be `[]`)	Empty list valid for pure-doc changes.
`risks`	yes	Use `["none"]` if there are genuinely none.
`recommended_next_agent`	yes	`agent: "none"` if work is complete.
`do_not_pass_downstream_without_verification`	yes (may be `[]`)	Cascading-hallucination defense.

Refusal

If the inbound handoff fails the field requirements, the specialist responds with ONLY this block, does no other work, and waits for re-issue:

```
return:
  schema_version: 1
  handoff_id: <copy from inbound, or "missing" if absent>
  from_agent: <specialist-name>
  to_agent: <orchestrator-name>
  status: needs_clarification
  reason: handoff_schema_invalid
  missing_or_invalid_fields:
    - field: <field name>
      problem: <what's wrong — missing, vague, no evidence, etc.>
  required_action:
    - <what the orchestrator must add or fix before re-issuing>
```

Failure-condition observation rule

If at any point during the work the specialist observes the inbound `failure_condition`, it STOPS immediately and responds with the return schema using `status: claim_rejected` (or `needs_clarification` if ambiguous). It cites the triggering evidence in `evidence_used`. It does NOT push through a falsified premise.

Worked examples

Example 1 — Backend bug investigation

Outbound (orchestrator → backend-api):

```yaml
handoff:
  schema_version: 1
  handoff_id: h-2026-05-02-bug-001
  from_agent: acme-orchestrator
  to_agent: backend-api
  goal: Find why /api/orders returns 500 for orders with > 100 line items.
  task_class: bug
  affected_roles: [customer]
  claims:
    - claim: The 500 originates in the order serialization step.
      evidence: src/api/orders/handler.ts:88
      confidence: high
    - claim: Order serialization runs synchronously and may exceed function timeout.
      evidence: instruction:.github/instructions/api.instructions.md
      confidence: medium
  verify_before_acting:
    - That src/api/orders/handler.ts:88 is still the serialization line.
    - That the timeout is actually exceeded (logs / metric, not just suspected).
  failure_condition: If logs show the 500 happens in the database query rather than serialization, STOP — the orchestrator's hypothesis about
      the failure location is wrong.
  in_scope:
    - src/api/orders/handler.ts
    - src/api/orders/serializer.ts
  out_of_scope:
    - The order schema (database concern; separate specialist if migration needed).
  instructions_to_read:
    - .github/instructions/api.instructions.md
    - .github/instructions/observability.instructions.md
  expected_artifacts:
    - Root cause statement (file:line)
    - Smallest fix or "no bug — load test issue" verdict
  hop_context:
    parent_task: Customer reports cart > 100 items always errors at checkout.
    previous_specialist: none
    previous_findings_summary: n/a
```

Return (backend-api → orchestrator):

```yaml
return:
  schema_version: 1
  handoff_id: h-2026-05-02-bug-001
  from_agent: backend-api
  to_agent: acme-orchestrator
  status: claim_rejected
  verified_claims:
    - claim: src/api/orders/handler.ts:88 is the serialization line.
      evidence: src/api/orders/handler.ts:88 (confirmed via codebase search + read)
  rejected_claims:
    - claim: The 500 originates in serialization.
      evidence: Stack trace from production logs shows the failure is in the database query at src/db/repositories/orderRepo.ts:142, not
        serialization. The query loads ALL line items in a single SELECT without pagination, exceeding the 30s function timeout for orders
        with > ~140 line items.
  unverified_claims: []
  evidence_used:
    - src/api/orders/handler.ts:80-95
    - src/db/repositories/orderRepo.ts:140-150
    - usages search for "loadOrderLineItems" in src/
    - production logs (sampled 3 failing requests)
  files_changed: []
  tests_run: []
  risks:
    - Real fix requires pagination of the query. Out of this handoff's scope.
  recommended_next_agent:
    agent: database
    why: Pagination of orderRepo.loadOrderLineItems is the correct fix.
  do_not_pass_downstream_without_verification:
    - That the timeout value is actually 30s (read from env or config; I inferred from observed behavior, did not check).
```

Example 2 — Mobile feature build

Outbound (orchestrator → mobile-ui):

```yaml
handoff:
  schema_version: 1
  handoff_id: h-2026-05-02-feat-002
  from_agent: acme-orchestrator
  to_agent: mobile-ui
  goal: Add a pull-to-refresh gesture on the OrderHistoryScreen.
  task_class: feature
  affected_roles: [customer]
  claims:
    - claim: OrderHistoryScreen lives at src/screens/OrderHistoryScreen.tsx.
      evidence: src/screens/OrderHistoryScreen.tsx
      confidence: high
    - claim: The app uses React Native's RefreshControl for pull-to-refresh elsewhere.
      evidence: instruction:.github/instructions/mobile-ui.instructions.md
      confidence: high
  verify_before_acting:
    - That OrderHistoryScreen uses a ScrollView or FlatList that can host RefreshControl.
  failure_condition: If OrderHistoryScreen renders a non-scrollable view (e.g. a Map or a custom virtualized list), STOP — the standard
      RefreshControl pattern won't apply and a different solution is needed.
  in_scope:
    - src/screens/OrderHistoryScreen.tsx
    - src/screens/__tests__/OrderHistoryScreen.test.tsx (add test)
  out_of_scope:
    - The order data fetching logic (already exists; just call refetch).
  instructions_to_read:
    - .github/instructions/mobile-ui.instructions.md
  expected_artifacts:
    - Edit to OrderHistoryScreen.tsx adding RefreshControl
    - New test asserting onRefresh triggers refetch
  hop_context:
    parent_task: Product wants pull-to-refresh on order history per ACME-1234.
    previous_specialist: none
    previous_findings_summary: n/a
```

Example 3 — Compliance review (REVIEW-ONLY specialist)

Outbound (orchestrator → legal-compliance):

```yaml
handoff:
  schema_version: 1
  handoff_id: h-2026-05-02-rev-003
  from_agent: acme-orchestrator
  to_agent: legal-compliance
  goal: Review the new email opt-in flow for GDPR/CASL/CAN-SPAM compliance before launch.
  task_class: review
  affected_roles: [customer]
  claims:
    - claim: The opt-in flow records consent timestamp + IP + opt-in source.
      evidence: src/db/migrations/2026-05-01-add-consent-log.sql
      confidence: high
  verify_before_acting:
    - That the migration has been applied to staging.
    - That the consent log columns are actually written by the new opt-in handler.
  failure_condition: If the consent log table exists but the opt-in handler does NOT write to it, STOP — the consent capture is non-functional
      and the compliance review is moot.
  in_scope:
    - REVIEW-ONLY. No code edits.
    - src/email/**
    - src/db/migrations/2026-05-01-*
  out_of_scope:
    - Any code changes (escalate to email-templates or backend-api specialists).
  instructions_to_read:
    - .github/instructions/security-privacy.instructions.md
  expected_artifacts:
    - Severity-ranked findings (blocker / high / medium / low)
    - Attorney-checklist of questions for counsel
  hop_context:
    parent_task: Pre-launch gate for new email opt-in feature.
    previous_specialist: none
    previous_findings_summary: n/a
```

Optional fields added in v1.2.0 (additive — `schema_version` stays 1)

Outbound, for multi-repo workspaces (Chapter 12) — omit in single-repo projects:

```yaml
  repo: <repo name from workspace.json — the ONLY repo this handoff may edit>
  contract_impact:
    level: <none | additive | breaking>
    contracts: [<names from CONTRACTS.md>]
    consumers_to_update: [<repo names — required when level != none>]
```

Outbound, any project — the evidence-confidence class from Chapter 11 may be used in place of the three-value `confidence:` on a claim (`verified-code` / `verified-schema` / `verified-test` / `verified-git` / `documented-unverified` / `historical` / `unknown`). Specialists treat anything that is not `verified-*`

exactly as they treat `confidence: low`.

Inbound, for multi-repo workspaces:

```
contracts_changed:
  - contract: <name>
    change: <one line>
    backward_compatible: <true | false>
    consumers_grepped: [<repo>:<path>, …]
```

Inbound, any project — `deferred_work:` lists anything the specialist is *not* doing that someone must (each item with the backlog file it was appended to). A return that names deferred work without a backlog path is incomplete (Chapter 11).

Versioning

- `schema_version: 1` is current.
- Additive changes (new optional fields) keep version 1.
- Breaking changes (renaming fields, changing field semantics) bump the version.
- Bumps require updating HANDOFF_SCHEMA.md, the orchestrator, and every specialist file in the same PR.

Why this schema is enough (hooks are optional, not required)

The schema is enforced by: 1. The orchestrator's instructions to ALWAYS emit the outbound block. 2. Every specialist's instructions to refuse delegations missing the block. 3. The orchestrator's instructions to consume the return block before merging work.

This is documentation enforcement — agents follow the rules because their instructions tell them to. It works because: - The Copilot harness at least enforces the `tools:` allowlist (so a misbehaving agent can't write where it shouldn't) - The orchestrator catches return-block violations (missing fields, wrong types) - Refusal is a one-shot rejection: if the specialist returns the refusal template, the orchestrator must re-issue with the missing fields

Copilot now has `preToolUse` / `agentStop` hooks (v1.2, `docs/10-MECHANICAL-ENFORCEMENT.md`), so a hook that validates the outbound YAML block before an `agent` tool call is possible — but the documentation enforcement still covers the majority of value at minimal complexity, and the framework ships no schema-validation hook by default.

05 — Instructions, Skills and Prompts

Three patterns for capturing knowledge that doesn't belong in agents: **instruction files** (path-globbed invariants), **agent skills** (repeatable workflows that work on every Copilot surface), and **prompt files** (IDE-only repeatable prompts invoked via slash command).

> v1.0/v1.1 of this chapter described prompt files as "the Copilot equivalent of skills". That was wrong: Agent Skills are a documented, cross-surface primitive and are the equivalent of Claude Code skills; prompt files are an IDE convenience. Corrected in v1.2.0 (verified 2026-08-22).

Instruction files

Path-globbed invariants. Each file in `.github/instructions/` has YAML frontmatter declaring which file paths it applies to. When Copilot is editing a matching file, the instruction file is auto-loaded into the prompt.

This is the **direct equivalent of "rules"** in other frameworks — and arguably cleaner because the path globbing is in YAML frontmatter rather than in body text.

When to write an instruction file

Write one when: - A bug shipped to production once and must not recur - A pattern is non-obvious from reading the code (e.g. "this column looks like a string but it's a JSON-encoded string in legacy rows") - A constraint crosses multiple files but isn't enforced by types/tests - An anti-pattern is tempting but wrong

Don't write one for: - Things obvious from the code - Style preferences (lint config does that) - One-off decisions (PR descriptions do that) - Anything covered by an existing instruction file (extend the existing one)

Instruction file structure

```
---
applyTo: "src/api/**/*.ts"
excludeAgent: "code-review"
---

# <Domain Name> Instructions

## Verify your globs — they fail silently, in both directions

A scoped rule is two claims: the FIELD name the tool reads, and the GLOBS inside it. Both fail
without a warning, and the two tools fail opposite ways.

**The field.** Copilot reads `applyTo:`. An instruction file with no `applyTo` is **not applied automatically at all** — the opposite of
        Claude Code, where an unscoped rule loads every session. Same mistake, opposite damage: on Claude you drown in context, on Copilot the
        guidance simply never arrives, and the output looks like a model ignoring your standards.

**The globs.** Glob syntax has already claimed `[` (character class) and `(` (group). A directory
literally named `[id]` written as `[id]` matches a single character, `i` or `d`. One named
`(admin)` written as `(admin)` matches nothing at all. Several mainstream web frameworks use
exactly those characters as their routing convention, so a project on one will write dead globs
naturally. Escape them:
```

applyTo: "src/app/\(marketing\)/**/*.tsx,src/app/api/users/\[id\]/route.ts"

```
**Neither failure is visible in review** — both look like ordinary paths. Copy
`templates/verify-rule-globs.mjs.template` into your scripts directory and run it whenever a glob
changes: it asserts every scoped file matches at least one real tracked file, and fails when two
matchers disagree about the same pattern. On a real migration it caught 5 bracket breaks and 13
parenthesis breaks across 7 files, one of which the author had not predicted.

**A glob that matches nothing is indistinguishable from a rule you never wrote.**

## Hard rules

### <Rule title — short imperative>

**Why:** <one-sentence reason — usually a past incident or invariant>

**How to apply:** <2-4 sentences — what to do, what helper to use, what to check>

### <Next rule>

...

## What NOT to do

- Bullet list of anti-patterns

## Cross-links

- `<related orientation map>`
- `<related canonical doc>`
- `<related instruction file>`
```

Frontmatter fields

Field	Required	Notes
`applyTo`	yes	Glob pattern. Multiple patterns separated by commas: `"**/*.ts,**/*.tsx"`.
`excludeAgent`	no	Prevent specific Copilot features from using this. Values: `"code-review"`, `"cloud-agent"`.

Glob patterns

Standard shell globs. Examples:

```yaml
applyTo: "src/api/**/*.ts"
applyTo: "src/db/migrations/**/*.sql"
applyTo: "src/{lib,utils}/auth-*.ts"
applyTo: "components/payment/**/*.{tsx,jsx}"
applyTo: "ios/Runner/**/*.swift"
applyTo: "lib/screens/**/*.dart"
applyTo: "internal/handlers/**/*.go"
applyTo: "app/models/*.rb"
```

Globs are matched against the relative path from project root.

Size budget (and code review)

The current docs give two limits (verified 2026-08-22): the code-review tutorial says to **limit any single instruction file to about 1,000 lines**, and the repository-instructions page says **instructions must be no longer than 2 pages**. The earlier "code review reads only the first 4,000 characters" sentence is no longer on the docs and is not a cap you should design around (Pitfall 2).

Still front-load the rules that matter most for code review — the reader under the tightest budget sees the top of the file first — but treat that as good practice, not a hard limit. When a file grows toward the 2-page mark, split it by domain.

To exclude an instruction from code review entirely:

```yaml
applyTo: "src/api/**/*.ts"
excludeAgent: "code-review"
```

Examples by tech

Backend / API

```
---
applyTo: "src/api/**/*.ts,src/server/**/*.ts"
---

### Always validate request body before reading database

**Why:** Three production incidents traced to unvalidated body data flowing to SQL parameters.

**How to apply:** Use `validateRequest(schema)` middleware in `src/lib/validation.ts`. Never read `req.body.<field>` directly in a handler.
```

Database / migrations

```
---
applyTo: "**/migrations/**/*.sql,prisma/schema.prisma"
---

### Migration order is sacred — never reorder, never edit applied migrations

**Why:** Migrations apply in filename order. Editing an applied migration silently diverges staging from production.

**How to apply:** New migrations get the next sequential timestamp prefix. Bug fixes to applied migrations require a NEW migration that
        supersedes the old behavior.
```

Frontend / UI

```
---
applyTo: "src/components/**/*.{tsx,jsx},src/app/**/*.{tsx,jsx}"
---

### Never trust component props for security checks — re-validate on the server

**Why:** A user can edit any prop via React DevTools.

**How to apply:** Server-side handlers re-validate authorization. Props are for rendering, not authorization.
```

Mobile

```
---
applyTo: "src/screens/**/*.tsx,src/components/**/*.tsx"
---

### iOS auto-zooms on input focus when font-size < 16px

**Why:** iOS Safari WebView feature; jarring UX.

**How to apply:** Set `font-size: 16px` minimum on all `<input>`, `<select>`, `<textarea>`. For visual size adjustment, scale via
        padding/transform, not font-size.
```

Anti-patterns in instruction files

❌ **Long rationale paragraphs.** Rules should be ≤ 4 sentences each. Move long stories to the canonical doc.

❌ **Aspirational rules.** "We should always…" — if it's not enforced, it's not a rule.

❌ **Rules that contradict other instruction files.** Have the context-librarian agent reconcile.

❌ **Files with no `applyTo`.** Without the glob, Copilot has no signal to load it.

❌ **Putting too many rules in one file.** Group by domain, not by "all rules in one file." If a file passes ~150 lines, split it — well inside the documented ~2-page budget.

Agent skills

Repeatable multi-step workflows that load on **every** Copilot surface. This is the Copilot equivalent of Claude Code skills, and the home for anything the cloud agent or the CLI must be able to run.

Location

- Project: `.github/skills/<skill-name>/SKILL.md` (one directory per skill). Copilot also discovers `.claude/skills/` and `.agents/skills/`.
- Personal: `~/.copilot/skills/`.

Frontmatter

Field	Required	Notes
`name`	yes	Lowercase with hyphens; matches the directory name.
`description`	yes	One sentence — when to use this skill. Drives automatic loading by relevance.
`license`	no	
`allowed-tools`	no	Tools the skill may use.
`argument-hint`	no	VS Code only.
`user-invocable`	no	VS Code only — whether `/skill-name` is offered to the user.
`disable-model-invocation`	no	VS Code only — prevents automatic loading.

Invocation

Type `/<skill-name>` in chat, or let Copilot load the skill automatically when the task matches its `description`.

Surfaces

Supported by the cloud agent, code review, the Copilot CLI, and agent mode in VS Code and JetBrains. **Prompt files are IDE-only** — VS Code's own guidance for a prompt that agents on the Agent Host don't pick up is "convert it to an agent skill". If a workflow must run from an issue assignment or from `copilot -p` in CI, it is a skill.

Skill structure

```
---
name: <skill-name>
description: <one sentence — when to use this>
---

# <Skill title>

<one paragraph — what this procedure guarantees>

## Steps

### 1. <Step name>
<2-4 sentences>

### 2. <Next step>
...

## Definition of Done

1. <Artifact>
2. <Verification>
```

When to write a skill

Write one when: - The same kind of task arrives 3+ times - The task has multiple steps with verification between them - The cost of skipping a step is high (e.g. forgot to run security review before launching a new auth flow) - The workflow must also run on the cloud agent or the CLI, not only in an IDE chat

Don't write one for: - One-off tasks - Tasks where the steps vary too much per instance - Things that are really just a single agent invocation

Common skills (most projects benefit from these)

- `.github/skills/<project-slug>-engineering/SKILL.md` — the six-gate playbook with the evidence ladder (`templates/engineering-playbook-skill.md.template`, Chapter 11). Invoke when no more specific skill fits.
- `.github/skills/investigate-bug/SKILL.md`, `.github/skills/build-feature/SKILL.md`, `.github/skills/qa-flow/SKILL.md`, `.github/skills/compliance-review/SKILL.md`, `.github/skills/context-refactor/SKILL.md` — the same five workflows listed under prompt files below, as skills so they also run on the cloud agent and CLI.

Prompt files

IDE-only repeatable prompts invoked via slash command. Each file in `.github/prompts/` has frontmatter + body. They are a convenience for workflows you only ever start by hand in VS Code / Visual Studio / JetBrains; the cloud agent and the Copilot CLI do not load them.

When to write a prompt file (rather than a skill)

Write one when: - The workflow is only ever started by a person in the IDE, and needs `${input:…}` prompting - You want a quick checklist without the directory-per-skill structure

Prefer a skill when: - The workflow must run on the cloud agent or the CLI - The workflow should load automatically when relevant, not only on `/name`

Don't write either for: - One-off tasks - Tasks where the steps vary too much per instance - Things that are really just a single agent invocation

Prompt file structure

```
---
description: <one-sentence description of when to use this — appears in /command picker>
---

<the prompt body — instructions, may include ${input:variable:hint} placeholders>

## Steps

### 1. <Step name>

<2–4 sentences>

### 2. <Next step>

...

## Definition of Done

1. <Artifact>
2. <Verification>
```

Frontmatter fields (current public preview)

Field	Required	Notes
`description`	yes	One sentence shown in the `/command` picker.
`agent`	optional	Specific agent to associate with this prompt.

(More fields may be supported by your IDE — check the Copilot prompt files documentation.)

Input placeholders

Inside the body, use `${input:NAME:HINT}` for prompted inputs:

```
Explain the following code in a clear, beginner-friendly way:

Code to explain: ${input:code:Paste your code here}
Target audience: ${input:audience:Who is this explanation for?}
```

When invoked via `/<filename>`, Copilot prompts the user for each input.

Invocation

In Copilot Chat, type `/` to see the picker. Pick from the list, or type the prompt name: `/<filename-without-extension>`. Example: `.github/prompts/investigate-bug.prompt.md` → `/investigate-bug`.

Available in: VS Code, Visual Studio, JetBrains. Currently in **public preview** — features may evolve.

Common prompt files (most projects benefit from these)

`.github/prompts/investigate-bug.prompt.md`

Steps: restate bug → identify affected roles → read minimum docs → trace UI → state → API → DB → tests → match touched paths to instruction files → reproduce → classify severity → propose smallest fix → verify.

`.github/prompts/build-feature.prompt.md`

Steps: restate feature → identify roles → read minimum docs → run pre-feature checklist → write acceptance criteria per role → identify instruction-covered paths → plan smallest implementation → implement incrementally.

`.github/prompts/qa-flow.prompt.md`

Steps: identify flows in scope → read testing strategy + relevant orientation map → cover golden path + edge cases + cross-role → verify functional + data correctness → severity-rank findings.

`.github/prompts/compliance-review.prompt.md`

Steps: identify data involved → identify if regulated subjects involved → read compliance orientation map → apply standing checklist → flag risks per regulation → produce attorney checklist.

`.github/prompts/context-refactor.prompt.md`

For maintaining the framework itself: read current INDEX → categorize suspect content → move to correct home → consolidate duplicates → flag conflicts → keep `.github/copilot-instructions.md` small.

Instructions vs skills vs prompt files vs agents – when to use which

If the knowledge is…	Put it in…
A path-scoped invariant ("don't do X when editing Y")	Instruction file
A multi-step workflow that recurs, on any surface (cloud agent, CLI, IDE)	Agent skill
A multi-step prompt only ever started by hand in the IDE, with `${input:…}`	Prompt file
A persona with its own scope and Definition of Done	Custom agent
A one-time decision rationale	PR description
Long-form architecture explanation	Canonical doc
Quick orientation for an area	Orientation map (`docs/ai-context/<area>.md`)
What is true right now / what we learned / what we deferred	`PROJECT.md` / `LEARNINGS.md` / `docs/<AREA>_BACKLOG.md` (Chapter 11)

When something feels like it could be 2 of those: prefer instruction file > skill > prompt file > agent (in that order). Instruction files are most precise (path-globbed, auto-loaded). Skills are next (named workflow, every surface, auto-loadable). Prompt files are IDE-only. Agents are heaviest (full persona).

Naming and lifecycle

Instruction files

- Lowercase with hyphens, `.instructions.md` suffix
- File name = domain name
- New gotcha from a new domain: create new file
- New gotcha from existing domain: append to existing file
- Stale instruction (the underlying constraint was fixed): delete it, don't leave it

Skills

- Lowercase with hyphens; directory name = `name:` = slash command (`.github/skills/investigate-bug/SKILL.md` → `/investigate-bug`)
- Keep `description` precise – it is what drives automatic loading; a vague one loads the skill on unrelated tasks
- Used < 3x per quarter and never auto-loaded: consider deleting; the workflow probably wasn't recurrent enough

Prompt files

- Lowercase with hyphens, `.prompt.md` suffix
- Filename becomes the slash command (`investigate-bug.prompt.md` → `/investigate-bug`)
- A skill and a prompt file with the same name are confusing – keep one
- Used < 3x per quarter: consider deleting; the workflow probably wasn't recurrent enough

06 — Invocation Modes

GitHub Copilot has more invocation modes than typical AI tools because it spans inline completions, Chat (in IDE and on github.com), Cloud Agent (autonomous), and CLI. Each surface has different rules for what gets loaded.

The seven modes

Mode 1: Inline completions (autocomplete)

> **Not re-verified on 2026-08-22.** Whether inline completions consult `.github/instructions/*.instructions.md` (as opposed to only `.github/copilot-instructions.md`) was not part of the v1.2/v1.1 verification pass and the two editions of this framework historically disagreed. Treat the bullet below as `documented-unverified` (Chapter 11/12) until you confirm it against the current Copilot docs for your IDE.

How: ghost text in the editor as you type.

Loads: - `.github/copilot-instructions.md` (auto) - `.github/instructions/<NAME>.instructions.md` matching the current file's path (auto)

Does NOT load: - Custom agents (`.github/agents/`) - Skills or prompt files - Hooks

Use for: typing assistance. Keep `.github/copilot-instructions.md` short for fast inline completions.

Mode 2: Chat in IDE (default)

How: Copilot Chat sidebar in VS Code / JetBrains / Visual Studio.

Loads: - `.github/copilot-instructions.md` (auto) - `.github/instructions/<NAME>.instructions.md` matching files in current chat context (auto)

Available on demand: - Custom agents (selectable from agent dropdown or @-mention) - Skills (`/<name>`, or auto-loaded by relevance) - Prompt files (invokable via `/<name>`; IDE-only) - Hooks from `.github/hooks/*.json` (VS Code also reads `.claude/settings.json` hooks) — Chapter 10

(Custom chat modes are retired — rename `.chatmode.md` files to `.github/agents/<name>.agent.md`, Pitfall 7.)

Use for: most interactive work. Combine `@<specialist>` + `/<workflow>` for power.

Mode 3: Chat with a specific agent (specialist mode)

How: Select a specific agent from the agent dropdown, OR type `@<agent-name>` in chat.

Loads: - The selected agent's `.github/agents/<name>.agent.md` body as system prompt - The agent's `tools:` allowlist is enforced - `.github/copilot-instructions.md` (still auto-loaded) - `.github/instructions/` matching paths (still auto-loaded)

Use for: narrow domain work. Examples: - `@frontend-ui` for UI session - `@backend-api` for API work - `@qa-functional` for QA pass with browser MCP - `@legal-compliance` for review-only session

The agent's `tools:` allowlist physically restricts what it can do (e.g. `legal-compliance` literally cannot edit files because it has no edit tools).

Mode 4: Chat with the orchestrator (multi-domain mode)

How: Select your `<project>-orchestrator` agent OR type `@<project>-orchestrator <task>` in chat.

Loads: the orchestrator's body as system prompt + everything from Mode 3.

Use for: - Anything spanning more than one role - Anything spanning more than one feature area - Data integrity, payments, security/privacy, compliance work - Bug investigations where root cause may cross layers - Feature builds where acceptance criteria need writing before implementation - Pre-release QA across multiple roles - Anything where the scope is ambiguous

The orchestrator delegates to specialists by `@<specialist>` mentions in its responses (you may see it propose to invoke a specialist; confirm or paste the recommended next step). In VS Code the orchestrator can also invoke a specialist directly as a subagent, restricted to the names in its `agents:` frontmatter (Chapter 3).

Mode 5: Cloud Agent (autonomous)

How: Assign a Copilot agent to a GitHub issue or PR. The cloud agent runs autonomously on github.com infrastructure, makes changes, and opens a PR for review.

Loads: - `.github/copilot-instructions.md` (auto) and `AGENTS.md` (nearest in the directory tree wins) - `.github/instructions/<NAME>.instructions.md` matching files it edits (auto) - The custom agent assigned (from `.github/agents/<NAME>.agent.md`) - Skills from `.github/skills/*/SKILL.md` - Hooks from `.github/hooks/*.json` — and **only** from there (Chapter 10) - For org/enterprise: org-level custom agents from `/agents/` in the org's `.github` or `.github-private` repository

Does NOT load: prompt files (IDE-only — convert to a skill).

Strong context isolation: the cloud agent runs in a fresh environment per task. No leakage from previous sessions. **Per-repository:** "Copilot cannot make changes across multiple repositories in one run" — one branch, one PR per task (Chapter 12 for cross-repo work).

Use for: well-defined, self-contained tasks where you'd otherwise hand off to a junior engineer. Not for ambiguous work that needs back-and-forth.

Mode 6: Copilot CLI, headless (`copilot -p`) — the current CLI

How: the agentic `copilot` command in the terminal. Interactive by default; non-interactive with `copilot -p "<prompt>"`.

Loads (from the working directory): `.github/copilot-instructions.md`, instruction files, `AGENTS.md` / `CLAUDE.md`, custom agents, skills, and hooks (`.github/hooks/*.json` plus `~/.copilot/hooks/`). There is **no `--cwd` flag** — run it from the target directory (or worktree). `--add-dir=<path>` grants access to additional directories.

Flags that matter for orchestration: - `--agent=<name>` — run as a specific custom agent (the orchestrator, for a scripted multi-domain task). - `--allow-tool=<tool>` / `--deny-tool=<tool>` / `--allow-all-tools` — tool permissions for a non-interactive run. A headless run that needs to edit or run commands must be granted them explicitly; a review-only agent should be run with nothing beyond read tools allowed. - `-s` / `--silent` — suppress non-output chatter for scripting. - `--output-format` is **not documented** — parse the plain text output, or have the agent write a file.

Use for: CI jobs, scheduled sweeps, and scripted delegation (a workspace orchestrator calling a repo's orchestrator — Chapter 12). Because hooks load here, a `copilot -p` run is still governed by the build-gate / doc-freshness hooks the repository ships.

Mode 7: `gh copilot suggest` / `explain` — the older CLI extension

How: `gh copilot suggest` / `gh copilot explain` via the GitHub CLI extension.

Loads: its own instruction system, separate from the repository's `.github/` customizations. It does not run agents, skills or hooks.

Use for: one-line shell command suggestions and explanations only. For anything agentic, use Mode 6.

Decision tree

```
Is this an editor/typing assist task?
  YES → Inline completions (no choice — that's what's running)
  NO ↓

Is this a quick question about code (read-only)?
  YES → Plain Chat (no agent selected)
  NO ↓

Is this narrow single-domain work (one specialist's territory)?
  YES → Chat with that specialist agent (@<specialist>)
  NO ↓

Is this cross-domain work, ambiguous scope, or production-sensitive?
  YES → Chat with orchestrator (@<project>-orchestrator)
  NO ↓

Is this a well-defined, self-contained task you'd assign to a junior?
  YES → Assign issue to Cloud Agent
  NO ↓

Is this a scripted / CI / scheduled run of a defined workflow?
  YES → copilot -p "…" --agent=<agent> from the target directory
  NO ↓

Is this a one-line terminal question?
  YES → gh copilot suggest / explain
```

Why we don't put the orchestrator persona in `.github/copilot-instructions.md`

The repository-wide instructions file is auto-loaded into: - Inline completions (every keystroke!) - Chat sessions (every prompt) - Code review (PR diff comments) - Cloud agent (every task)

Putting the orchestrator's full delegation persona there would: - **Slow down inline completions** (every keystroke loads the persona) - **Pollute code review** with delegation instructions ("delegate this fix to backend-api") - **Force every interaction** through the orchestrator's "delegate, don't implement" pattern

Instead, `.github/copilot-instructions.md` is a **thin router** — golden rules + workflow + cross-links. The orchestrator persona is in `.github/agents/<project>-orchestrator.agent.md` and only loads when explicitly selected.

Mode comparison table

Mode	Loads orchestrator persona?	Specialists invokable?	Tool allowlist enforced?	Context isolation
Inline completions	No	No	N/A	n/a
Plain Chat	No	Yes (manual `@`)	Default Chat tools	Within Chat session
Specialist Chat	No	(just this one)	Yes (per agent's `tools:`)	Within Chat session
Orchestrator Chat	Yes	Yes (via orchestrator's `@` calls)	Yes (per agent's `tools:`)	Within Chat session
Cloud Agent	If assigned	If multi-agent	Yes (per agent's `tools:`)	Strong (fresh env)
Copilot CLI (`copilot -p`)	If `--agent=`	Yes (agents load)	Yes (`tools:` + `--allow-tool` / `--deny-tool`)	Fresh per run
`gh copilot suggest`	n/a (separate)	n/a	n/a	Separate context

Practical recipe

A well-functioning team uses several modes:

```
# Quick fix (90% of casual work)
Open Copilot Chat, no agent selected
"Update the button label from 'Sign In' to 'Sign in'"

# Specialist deep-dive (focused domain work)
Select @frontend-ui in Chat
"Audit our Suspense boundaries for client-component leaks"

# Full orchestrated work (multi-domain, production-sensitive)
Select @<project>-orchestrator in Chat
"Add a new payment method that requires KYC review and cron-driven activation"

# Autonomous bug fix (well-defined, junior-level)
Assign GitHub issue to Copilot
The cloud agent investigates, fixes, opens PR

# Scripted run (CI, cron, cross-repo delegation)
cd <repo>   # no --cwd flag — the CLI loads customizations from the working directory
copilot -p "Run /qa-flow on the checkout page and write findings to docs/qa/latest.md" --agent=qa-functional --allow-tool=<read/run tools as
needed> -s
```

Scheduled runs — standing routines (v1.3)

Modes 5 and 6 put on a clock, with governance. A routine is a narrow charter file executed on a schedule — a GitHub Actions `schedule:` workflow that assigns the routine's issue to the cloud agent (Mode 5), or runs `copilot -p --agent=<routine-agent>` headless (Mode 6) — producing small PRs behind review gates, never direct writes. It is the only way this framework runs with nobody watching, which is why it carries the strictest output contract: repro + truth table on every fix-PR, one reporting surface, budgets and attempt caps, and a human on every merge. Full conventions, starter catalog and budgets: `docs/13-STANDING-ROUTINES.md`.

Anti-patterns

❌ **Using plain Chat for everything.** Cross-domain work flooded with context. Specialists never invoked.

❌ **Using `@<orchestrator>` for typos.** Forced delegation overhead. Specialist spawn for a one-line fix.

❌ **Not knowing which mode you're in.** Check the agent dropdown in Chat. If you're not sure, you're probably in the wrong mode.

❌ **Putting the orchestrator persona in `.github/copilot-instructions.md`.** See section above — it pollutes everything.

❌ **Letting the Cloud Agent take ambiguous tasks.** Cloud Agent works best on tasks with clear scope and acceptance criteria. Ambiguity → bad PRs → human cleanup.

When you change projects

Each project has its own `.github/agents/`, so `@<project>-orchestrator` is project-specific. The orchestrator name should be `<project>-orchestrator` (e.g. `acme-orchestrator`, not just `orchestrator`) so that when you have multiple repos open simultaneously, you can tell at a glance which orchestrator is active.

Surface support matrix (current)

Feature	VS Code	JetBrains	Visual Studio	github.com Chat	Cloud Agent	Code review
`.github/copilot-instructions.md`	✅	✅	✅	✅	✅	✅
`.github/instructions/*.instructions.md` (auto-load on path match)	✅	✅	✅	✅	✅	✅
`AGENTS.md` (nearest wins)	✅ root (nested behind `chat.useNestedAgentsMdFiles`)	not documented	not documented	not documented	✅	✅
`.github/skills/*/SKILL.md` (agent skills)	✅	✅	not documented	not documented	✅	✅
`.github/prompts/*.prompt.md` (slash commands, IDE-only)	✅	✅	✅	❌	❌	❌
`.github/agents/*.agent.md` (custom agents)	✅	✅	✅	✅	✅	–
`agents:` subagent allowlist	✅	not documented	not documented	❌	❌ (ignored)	–
`.github/hooks/*.json` (lifecycle hooks)	✅ (also `.claude/settings.json`)	not documented	not documented	not documented	✅ (`.github/hooks` only)	not documented
`.github/chatmodes/*.chatmode.md`	RETIRED – rename to `.agent.md`	RETIRED	RETIRED	❌	❌	❌
MCP servers per agent	✅	✅	✅	partial	partial	–

(Verified against docs.github.com and code.visualstudio.com on 2026-08-22. "not documented" means the current docs neither confirm nor deny it – do not assume either way. Surface support evolves rapidly; re-verify quarterly – Pitfall 20.)

07 — Folder Structure

How to organize Copilot config and documentation so the AI (and humans) always know where to look.

The Copilot config tier

```
.github/
├── copilot-instructions.md             ← repo-wide router (auto-loaded everywhere)
├── agents/                             ← orchestrator + specialists
│   ├── <project>-orchestrator.agent.md
│   └── <specialist>.agent.md
├── instructions/                       ← path-globbed invariants (frontmatter has applyTo:)
│   └── <domain>.instructions.md
├── skills/                             ← repeatable workflows (cross-surface) — v1.2
│   └── <name>/SKILL.md
├── prompts/                            ← repeatable workflows, IDE-only (slash commands)
│   └── <workflow>.prompt.md
└── hooks/                              ← optional mechanical enforcement — v1.2
    └── framework.json                  (chat modes are retired: rename .chatmode.md → .agent.md)
```

The documentation tier

```
docs/
├── ai-context/                         ← TIER 1: orientation maps (read by Copilot agents)
│   ├── INDEX.md
│   ├── PROJECT.md                      ← current truth: what is live where (Chapter 11)
│   ├── LEARNINGS.md                    ← decisions, failures, corrections (Chapter 11)
│   ├── GLOSSARY.md                     ← one name per concept (Chapter 11)
│   ├── HANDOFF_SCHEMA.md
│   ├── ORCHESTRATION_SPOONFEEDER.md
│   └── <area>-experience.md
│
├── ARCHITECTURE.md                     ← TIER 2: canonical references
├── API.md
├── TECH_STACK.md
├── <UPPERCASE>.md
├── <AREA>_BACKLOG.md                   ← deferred work, one per area (Chapter 11)
│
└── _archive/                           ← TIER 3: frozen historical
    ├── README.md
    └── <YYYY-MM>/
```

Tier 1 — Orientation maps (`docs/ai-context/`)

Purpose: Per-area guides Copilot reads at the start of any task in that area.

Format: 50-150 lines per file.

Audience: Copilot agents (primary), humans (secondary).

Naming: `<area>-experience.md` or `<area>-<concern>.md` . Examples: - `customer-experience.md` - `admin-experience.md` - `auth-and-sessions.md` - `payments.md` - `realtime-sync.md` - `data-pipeline.md`

Body sections: 1. **2-sentence orientation.** What this area is, what it isn't. 2. **Key file paths.** 3. **3-5 gotchas worth knowing.** 4. **Cross-links** to canonical refs and to related instruction files.

Special files: - `INDEX.md` — task-type → docs + agent map - `HANDOFF_SCHEMA.md` — bidirectional handoff schema - `ORCHESTRATION_SPOONFEEDER.md` — human-facing usage guide - `PROJECT.md` / `LEARNINGS.md` / `GLOSSARY.md` — the project-truth set (Chapter 11) - `MIGRATION_LEDGER.md` (optional) — tracks doc/structure migration progress - `LEGACY_BACKUP.md` (optional) — frozen pre-refactor snapshot

> **Why `docs/ai-context/` and not `docs/copilot-context/` ?** The orientation maps are tool-agnostic. If you later add Claude Code, Gemini Code Assist, or other AI tools, they'll all read from the same place. Don't tie this folder to a single AI vendor.

Tier 2 — Canonical references (`docs/<UPPERCASE>.md`)

Purpose: Authoritative source of truth. Full detail. Updated when underlying truth changes.

Format: As long as needed. No artificial line cap.

Audience: Humans (primary), agents (when deeper detail needed).

Naming: `UPPERCASE.md` . Convention signals "this is canonical."

Common files: `ARCHITECTURE.md` , `API.md` , `TECH_STACK.md` , `CHANGELOG.md` , `PRODUCT_REQUIREMENTS.md` , `BUSINESS_DOCUMENT.md` .

Cross-referencing: - Orientation maps in `ai-context/` link DOWN to canonical docs. - Canonical docs do NOT link UP to orientation maps. - Agents link to either, depending on depth needed.

Tier 3 — Frozen archive (`docs/_archive/`)

Purpose: Historical material no longer authoritative but worth preserving for audits.

Format: Whatever the original was — markdown, HTML, PNG, CSV.

Audience: Audits only. Active docs never link here.

Naming: Date-prefixed subdirectories: `<YYYY-MM>/<file>` .

Required: `_archive/README.md` documenting: - What this directory is - Three rules: don't link from active docs, don't delete, don't move back to root - "Known link rot" section if you migrated from a structure with cross-links

What goes where — decision tree

```
You have a doc to write or move. Where?

Is it a per-area gotcha guide of 50-150 lines?
  → docs/ai-context/<area>.md

Is it the system architecture, API reference, or another full-detail doc?
  → docs/<UPPERCASE>.md

Is it a path-globbed invariant ("don't do X when editing Y")?
  → .github/instructions/<domain>.instructions.md (with applyTo: in frontmatter)

Is it a multi-step workflow that recurs?
  → .github/skills/<name>/SKILL.md (every surface); only if it is started by hand in the IDE
    and needs ${input:…} prompting → .github/prompts/<workflow>.prompt.md

Is it an agent persona definition?
  → .github/agents/<name>.agent.md

Is it a sprint report, audit, post-mortem, or dated snapshot?
  → docs/_archive/<YYYY-MM>/<file>.md

Is it a one-time decision rationale?
  → PR description or commit message — NOT docs/

Is it just routing ("for X tasks, use Y agent")?
  → .github/copilot-instructions.md (kept SHORT) or docs/ai-context/INDEX.md
```

Folder-level conventions

Root level

The root contains ONLY: - Tech-stack config files (`package.json` , `tsconfig.json` , `Cargo.toml` , `go.mod` , etc.) - Build/deploy config (`Dockerfile` , `docker-compose.yml` , etc.) - CI config (`.github/workflows/` , `.gitlab-ci.yml`) - Test runner config (`playwright.config.ts` , `pytest.ini` , etc.) - The framework files (`AGENTS.md` if used, `.github/` , `docs/`) - Application directories (`src/` , `app/` , `lib/` , `pkg/` , etc.) - Static assets (`public/` , `assets/`)

The root does NOT contain: - Loose orphan scripts (move to `scripts/_archive/` if not actively used) - Screenshots or images (move to `docs/_archive/screenshots/` or `assets/`) - Backup env files (gitignore + delete from history if secrets were committed) - Generated artifacts (gitignore) - Stub directories with one file (consolidate elsewhere)

Tracked-cache cleanup

Common directories to gitignore (often tracked by accident): - IDE caches (`.vscode/` , `.idea/` — at minimum gitignore the auto-generated parts) - Test runner per-run state (`test-results/` , `coverage/`) - Database CLI temp dirs - Build artifacts (`.next/` , `dist/` , `build/` , `__pycache__/`) - Local logs (`logs/`)

Env file safety

Add broad gitignore patterns:

```
.env
.env.*
!.env.example
.env*.bak*
.env*staging_tmp
.env*tmp
```

If env files with secrets are already in git history, that's a separate workstream — rotate secrets first, then scrub history with `git-filter-repo` or BFG.

Variants for non-standard projects

Monorepo

```
monorepo-root/
├── .github/
│   ├── copilot-instructions.md         ← root router; mentions per-package files exist
│   ├── agents/                         ← orchestrator + cross-package specialists
│   ├── instructions/                   ← cross-package instructions
│   ├── skills/
│   └── prompts/
├── docs/                               ← cross-package docs
│   └── ai-context/
└── packages/
    ├── package-a/
    │   └── AGENTS.md                   ← package-specific notes
    └── package-b/
        └── AGENTS.md
```

(Note: nested `.github/` directories don't get loaded the same way – use `applyTo:` globs in root `.github/instructions/` to scope per-package.)

Multi-product in one repo

```
your-product-repo/
├── .github/agents/
│   ├── <project>-orchestrator.agent.md
│   ├── product-a-frontend.agent.md
│   ├── product-a-backend.agent.md
│   ├── product-b-frontend.agent.md
│   └── shared-database.agent.md
├── .github/instructions/
│   ├── product-a.instructions.md       (applyTo: "src/products/a/**")
│   └── product-b.instructions.md       (applyTo: "src/products/b/**")
├── docs/ai-context/
│   ├── INDEX.md
│   ├── product-a-experience.md
│   └── product-b-experience.md
```

Mobile + web shared

```
your-app/
├── apps/
│   ├── web/                            ← Next.js / Vite / etc.
│   └── mobile/                         ← React Native / Flutter / native
├── packages/                           ← shared libs
└── .github/agents/
    ├── web-ui.agent.md                 ← only edits apps/web/
    ├── mobile-ui.agent.md              ← only edits apps/mobile/
    ├── shared-libs.agent.md            ← only edits packages/
    └── backend-api.agent.md            ← stays separate if hosted elsewhere
```

Organization-level configuration

If you have GitHub Copilot Business or Enterprise:

```
your-org/
├── .github-private/                    ← special org-level repo
│   └── agents/
│       └── company-wide-agent.agent.md ← available across all org repos
└── (regular repos as above)
```

Organization custom instructions are configured in the GitHub UI at organization settings – not in files.

08 — Common Pitfalls

Thirty hard-won lessons. GitHub Copilot has its own customization quirks layered on top of generic multi-agent pitfalls. Read this before bootstrapping.

> Pitfalls 1–19 date from v1.0/v1.1. Pitfalls 20–28 were added in v1.2.0 after three further months of production use; Pitfalls 2, 7, 9 and 19 were rewritten in v1.2.0 because the platform facts they rested on changed (verified against the official Copilot and VS Code docs on 2026-08-22), and Pitfall 20 records which v1.0/v1.1 claims the platform has since made false. Pitfalls 29–30 were added in v1.3.0 alongside Chapter 13 (standing routines).

Pitfall 1: Putting the orchestrator's persona in `.github/copilot-instructions.md`

`.github/copilot-instructions.md` is auto-loaded into EVERY Copilot interaction — including inline completions and code review. Putting the orchestrator's full delegation prose there means: - Inline completions get slower (the persona loads on every keystroke) - Code review gets polluted with "delegate this to backend-api" instructions - Every plain Chat session forces the orchestrator pattern on simple tasks

Right answer: Keep `.github/copilot-instructions.md` short (under 200 lines). It's a router. The orchestrator persona lives in `.github/agents/<project>-orchestrator.agent.md` and only loads when explicitly selected.

Pitfall 2: Instruction files have a documented size budget — long files get ignored or skimmed

The current GitHub docs give two size limits for instructions (verified 2026-08-22): the code-review tutorial says to **limit any single instruction file to about 1,000 lines**, and the repository-instructions page says **instructions must be no longer than 2 pages**. Earlier versions of this framework (v1.0/v1.1) cited a "code review reads only the first 4,000 characters" rule; that sentence is **no longer on the current docs** and should not be quoted as a hard cap.

The practical failure is unchanged: a long instruction file dilutes the rules that matter, and whichever reader is under the tightest budget (code review, inline completions) sees the least of it.

Right answer: - Keep each `.github/instructions/<NAME>.instructions.md` well inside the documented budget (~2 pages; never anywhere near 1,000 lines). Split by domain when a file grows. - Front-load the rules that matter most for code review — good practice, not a hard cap. - Use `excludeAgent: "code-review"` for instructions that aren't review-relevant, so the review reader isn't spending its budget on implementation guidance. - Re-check the two size sentences quarterly (Pitfall 20); they have already changed once.

Pitfall 3: Forgetting `applyTo:` makes the file useless

An instruction file with no `applyTo:` field in its frontmatter doesn't get auto-loaded by anything — it's just an orphan markdown file.

Right answer: Every `.github/instructions/*.instructions.md` file MUST have a non-empty `applyTo:` glob in frontmatter. If you have rules with no clear path scope, they belong in `.github/copilot-instructions.md` (the auto-loaded router) instead.

Pitfall 4: `tools:` allowlist is your strongest defense — don't omit it

If you don't declare `tools:` in an agent's frontmatter, the agent inherits ALL tools available in the current Copilot session — including edit tools, terminal tools, and any installed MCP servers.

For REVIEW-ONLY agents (legal-compliance, security-privacy, architecture-review), this is a hidden disaster. The agent can write to your repo even though its instructions say it can't.

Right answer: Always declare `tools:` explicitly. For REVIEW-ONLY agents, list ONLY read-side tools. Verify by checking your IDE's agent dropdown — the agent should not be able to invoke edit tools.

Pitfall 5: Cloud Agent vs IDE Chat agent — different contexts

The same `.github/agents/<NAME>.agent.md` file works in both, BUT: - The cloud agent runs in true context isolation (fresh environment per task) - The IDE chat agent runs within the active Chat session's context — it can see prior messages

If your specialist's body assumes the cloud agent's isolation (e.g. "you have no prior context, start fresh"), it'll behave oddly in IDE chat where prior context exists.

Right answer: Write specialist bodies that work in BOTH contexts: - Don't assume isolation; the agent might inherit Chat context - Don't assume Chat context; the agent might run cleanly in cloud - Use explicit handoff schema (`handoff_id` , `previous_specialist` , etc.) so the agent isn't relying on implicit conversation memory

Pitfall 6: Prompt files are public preview — features may shift

`.github/prompts/*.prompt.md` is in **public preview** as of late 2025 / early 2026. Frontmatter fields and invocation behavior may change in future Copilot releases.

Right answer: Stick to documented frontmatter fields (`description` , `agent`). Avoid IDE-specific extensions you can't verify. When prompt files leave preview, check the release notes for any breaking changes.

Note also that prompt files are **IDE-only** (VS Code, Visual Studio, JetBrains) — the cloud agent and the Copilot CLI do not use them. A workflow that must run on those surfaces belongs in an Agent Skill (`.github/skills/<name>/SKILL.md` , Chapter 5), which is the cross-surface primitive; VS Code's own guidance for a prompt file that agents don't pick up is "convert it to an agent skill".

Pitfall 7: `.github/chatmodes/*.chatmode.md` is RETIRED — rename to `.agent.md`

Custom chat modes (`.chatmode.md`) were the earlier name for what are now custom agents. As of the current docs (verified 2026-08-22) chat modes are **retired, not "older but fine"**: the documented location is `.github/agents/<NAME>.agent.md` (a bare `.md` name still works; `.agent.md` is current), and existing `.chatmode.md` files should be **renamed** to `.agent.md` . The agent format carries the fields chat modes lacked (`target` , `mcp-servers` , `user-invocable` , `disable-model-invocation` , `metadata` , and in VS Code `agents` , `handoffs` , `argument-hint` , `hooks`).

v1.0/v1.1 of this framework shipped a `chatmode.md.template` and told you to "keep chat modes only if you have legacy files you don't want to migrate". Don't keep them.

Right answer: - New agents: `.github/agents/<NAME>.agent.md` . - Existing `.github/chatmodes/<NAME>.chatmode.md` : `git mv` to `.github/agents/<NAME>.agent.md` , keep the frontmatter (`description` , `tools` , `model` carry over), reload the IDE (Pitfall 15), and confirm the agent appears in the dropdown. - Don't create new `.chatmode.md` files for any reason.

Pitfall 8: Treating documentation enforcement as runtime enforcement

The handoff schema, the universal evidence rule, the `failure_condition` observation, the soft hop limits — these are **documentation discipline**. They work because agents follow their instructions. They do NOT physically prevent a misbehaving agent from emitting a malformed handoff.

What IS runtime-enforced (by the Copilot harness): - `tools:` allowlists per agent - `mcp-servers:` per-agent MCP scoping - `applyTo:` globs in instruction files (auto-loading conditional on path match) - `target:` field (environment scoping for `vscode` vs `github-copilot`) - `disable-model-invocation` and `user-invocable` (agent visibility/routing) - Body length cap (30,000 chars)

What is NOT runtime-enforced: - The handoff YAML block being present - The evidence rule being followed - Refusal of vague delegations - Hop limits ("3rd same-specialist delegation → escalate") - Definition of Done completeness

Also runtime-enforced, since v1.2.0 (Chapter 10): - `preToolUse` hooks in `.github/hooks/*.json` (`permissionDecision: allow | deny | ask` ; exit 2 = deny) - `agentStop` hooks that return `{"decision":"block"}` to refuse ending a turn - `agents:` subagent allowlists — VS Code only (Pitfall 9)

Right answer: Be honest about which layer enforces what. Documentation discipline catches ~95% of value at ~5% of complexity. If you need hard enforcement of a documentation rule, write a `preToolUse` hook that validates the handoff block before the agent call proceeds, or an `agentStop` hook that checks the return block — `docs/10-MECHANICAL-ENFORCEMENT.md` has the contract. A custom CI check remains the backstop for surfaces that don't load your hooks.

Pitfall 9: The agent-to-agent allowlist exists in VS Code and does not exist on the cloud agent

v1.0 of this chapter said "Copilot doesn't expose a per-agent allowlist for which other agents the orchestrator can invoke". That is now **half wrong** (verified 2026-08-22):

- **VS Code:** the `agents:` frontmatter property on a custom agent lists which agents it may invoke as **subagents** (`*` = all). That list **is** an allowlist — an agent not named cannot be invoked through the `runSubagent` tool. Nested invocation (a subagent invoking a subagent) is off unless `chat.subagents.allowInvocationsFromSubagents` is enabled.
- **Cloud agent (github.com):** GitHub's `agent` tool alias "allows a different custom agent to be invoked to accomplish a task", but the docs publish **no per-agent allowlist** for it. `agents:` is a VS Code property; the cloud agent ignores it. If the orchestrator invokes an unintended agent name there, there is no harness-level rejection.

So an `agents:` field on your orchestrator gives you real enforcement in one surface and documentation in the other. Write it either way — it costs one line and it is the only place the allowed specialist list is machine-readable.

Right answer: - Put `agents: [<specialist-1>, <specialist-2>, …]` on the orchestrator (VS Code allowlist; ignored by the cloud agent). Never `*` on an orchestrator — that reopens every built-in and plugin agent. - Keep the allowed specialist list in the orchestrator's body as well, so the cloud agent has the same routing table as prose. - Specialists still return `recommended_next_agent` rather than invoking each other — a flat tree is auditable and the handoff schema survives every hop (Chapter 3). - For the cloud agent, the only hard control is which agents exist at all: repository `.github/agents/` plus, on Business/Enterprise, org-level agents in the org's `.github` or `.github-private` repository.

Pitfall 10: MCP server auth lives in `${{ secrets.* }}`

Copilot MCP server config supports environment variable substitution from secrets:

```
mcp-servers:
  some-mcp:
    type: 'local'
    command: 'some-tool'
    env:
      API_KEY: ${{ secrets.SOME_MCP_API_KEY }}
```

Don't hardcode API keys in `.github/agents/*.md` files. Use the secrets reference syntax. Configure secrets at the user, repository, or organization level depending on scope.

Pitfall 11: `AGENTS.md` is a "lowest common denominator" file

`AGENTS.md` (or `CLAUDE.md`, `GEMINI.md`) is recognized by Copilot as a fallback/cross-AI metadata file. Per the docs, it's "currently not supported by all Copilot features."

Right answer: Don't put critical Copilot-specific config in `AGENTS.md`. Use `.github/copilot-instructions.md` for Copilot-specific routing. Use `AGENTS.md` only for shared cross-AI metadata that you want all AI tools (Copilot + Claude + Gemini) to read.

Pitfall 12: Inline completions only see the first ~few thousand chars of context

Inline completions are latency-sensitive. The model context for inline completions is intentionally smaller than Chat. Don't expect inline completions to know about complex orchestration patterns — they only see: - The current file's open content - Some surrounding files (depending on IDE configuration) - `.github/copilot-instructions.md` (if present, but truncated for latency) - Path-matching `.github/instructions/*.instructions.md` (truncated; whether inline completions load these at all is not re-verified — Chapter 6, Mode 1)

Right answer: Optimize `.github/copilot-instructions.md` for inline completions: lead with the most important constraints in the first 500 chars. The orchestration / agent routing details belong in agent files (which inline completions don't load anyway).

Pitfall 13: Ignoring `.env*` history

If env files with secrets are already in git history, removing them from the working tree doesn't remove the secrets from history.

Right answer: 1. Rotate every credential that was in any leaked env file. 2. Use `git-filter-repo` or BFG Repo-Cleaner to scrub history. 3. Force-push the scrubbed history (coordinate with the team). 4. Add broad gitignore patterns: `.env*.bak*`, `.env*staging_tmp`, `.env*tmp`.

Pitfall 14: Letting `docs/` rot without a librarian

Without active maintenance, `docs/` accumulates: sprint reports, audit reports referenced by no one, outdated decisions, etc. After a year, engineers can't tell which are authoritative.

Right answer: Have a `context-librarian` specialist whose job is exactly this. Schedule a quarterly cleanup pass. Move stale material to `docs/_archive/<YYYY-MM>/`.

Pitfall 15: Reorganizing `.github/` config without checking IDE behavior

If you rename `.github/agents/foo.agent.md` → `.github/agents/bar.agent.md` mid-task, IDE Copilot Chat may not pick up the change until you reload the IDE. The cloud agent picks up new agent files on the next assignment.

Right answer: - Reload your IDE after structural changes to `.github/` - Verify with the agent dropdown — the renamed agent should appear - For team-wide changes, communicate so others reload too - Avoid renames mid-PR; do structural changes in their own PR

Pitfall 16: One huge instructions file instead of scoped files

Putting all rules into `.github/copilot-instructions.md` defeats the purpose of `.github/instructions/`. Repository-wide instructions load on every interaction; path-globbed instructions load only when relevant.

Right answer: Refactor a long `.github/copilot-instructions.md` into: - Keep router content (golden rules + workflow + cross-links) in `.github/copilot-instructions.md` (target < 200 lines) - Move per-area gotchas to `.github/instructions/<domain>.instructions.md` (with `applyTo:` globs) - Move long architecture explanations to `docs/<UPPERCASE>.md` - Move sprint history to `docs/_archive/`

Pitfall 17: Bootstrap on a repo with existing Copilot config

If you ran VS Code's `/init` previously, or your team has been adding to `.github/copilot-instructions.md` for months, BOOTSTRAP-PROMPT.md must NOT silently overwrite that work.

Right answer: the prompt now includes mandatory pre-flight checks at the top — snapshot to `.github-pre-bootstrap-backup/`, naming-collision detection, `applyTo:` glob conflict detection, drift detection on existing instructions, and a decision gate that STOPS if any pre-flight raised a `<NEEDS USER CONFIRMATION>` flag.

Specific risks: - Existing `.github/copilot-instructions.md` with team rules → Pre-flight 4 (drift detection) flags stale content; Step 11 (merge step) shows a 3-pane diff before writing. - Existing `.github/agents/<name>.agent.md` (or `<name>.md`, or a `.chatmodes/<name>.chatmode.md`) with same name as a proposed specialist → Pre-flight 2 (naming collision check) STOPS for explicit user decision per file. - Existing `.github/instructions/` with overlapping `applyTo:` → Pre-flight 3 detects glob overlap; both files would load creating contradictory rules. - Existing `.github/chatmodes/` heavily used → Pre-flight 5 surfaces the parallel system; user decides migrate vs coexist.

The pre-flight workflow is non-optional — even on apparent greenfield projects, run it. Cost is 30 seconds; benefit is never silently destroying team work.

Pitfall 18: Forgetting that custom agents — and cloud-agent runs — are per-repository

`.github/agents/<NAME>.agent.md` is per-repo. If you have multiple repos and want shared agents, you have two options: - Copy agent files into each repo (drift risk) - For Business/Enterprise: define org-level agents under `/agents/` in the organization's `.github` or `.github-private` repository

Personal-tier Copilot doesn't support cross-repo agents.

The cloud agent is per-repository in a stronger sense too: per the docs, "Copilot cannot make changes across multiple repositories in one run" — one branch, one PR per task. A cross-repo change is several cloud-agent tasks (or an IDE/CLI session over several checkouts), never one.

Right answer: For multi-repo orgs on Business/Enterprise, define shared infrastructure at the org level. For personal/single-repo, just keep agents in the one `.github/`. For a cross-repo *workflow* (shared contracts, one orchestrator over several services), see `docs/12-MULTI-REPO-WORKSPACES.md`.

Pitfall 19: Assuming Copilot has no hooks — or assuming every surface loads them the same way

Retraction. The v1.1 text of this pitfall said "Copilot does not expose programmable lifecycle events" and told you to wire automation into the IDE or a pre-commit hook instead. That is **no longer true** (verified 2026-08-22). Copilot now has lifecycle hooks — `sessionStart`, `sessionEnd`, `userPromptSubmitted`, `preToolUse`, `postToolUse`, `agentStop`, `subagentStart`, `subagentStop`, `errorOccurred`, `preCompact`, `permissionRequest`, `notification` — configured as `.github/hooks/*.json` (plus `~/.copilot/hooks/` for personal hooks). `preToolUse` can return `permissionDecision: allow | deny | ask` (exit 2 = deny, fail-closed); `agentStop` can return `{"decision":"block","reason":"…"}` to force another turn. Every correction-capture / build-gate / lint-fix / doc-freshness pattern the Claude edition runs as a hook now has a Copilot mapping — **`docs/10-MECHANICAL-ENFORCEMENT.md` carries the per-pattern translation, the hook I/O contract and the templates**. Don't rebuild them from this paragraph.

The pitfall that survives is subtler — three ways a hook you installed silently isn't running:

- **The cloud agent loads only `.github/hooks/*.json`.** VS Code additionally reads Claude-style hooks from `.claude/settings.json` (PascalCase event names), and the CLI reads `~/.copilot/hooks/`. A hook that lives only in `.claude/settings.json` works on a laptop and is absent from every cloud-agent run. Put team hooks in `.github/hooks/`.
- **Timeouts fail OPEN.** A hook that exceeds `timeoutSec` (default 30) is treated as if it had allowed the action. A build-gate that sizes its build to the timeout is a gate that opens exactly when the build is slow — see Pitfall 25 and size the gate's *own* internal cap below the hook timeout.
- **The platform-native half is still native.** `applyTo:` auto-loading on `.github/instructions/*.instructions.md` remains the right tool for rule surfacing (zero scripting, can't fail silently). A `preToolUse` hook that re-implements it is a second, weaker copy.

Right answer: read `docs/10-MECHANICAL-ENFORCEMENT.md` before writing any hook. Install team hooks under `.github/hooks/`, verify each one fires on the surface you care about (cloud agent, CLI, VS Code are three separate checks), and treat a timeout as "inconclusive" in the hook's own logic rather than relying on the platform's fail-open default.

This pitfall most often hits teams running both editions of the framework: the documentation discipline (instruction files, agents, handoff schema) is shared, and the enforcement layer is now shared too — but the file locations and event names differ, so a hook ported by copy-paste lands in a directory one surface never reads.

Pitfall 20: The platform moves under your conventions — re-verify the docs every quarter

Three claims this framework made in v1.0/v1.1 became false within a few months, and one number it quoted disappeared from the docs entirely (all re-verified against docs.github.com and code.visualstudio.com on 2026-08-22):

- **"Copilot has no hooks."** It does — `sessionStart` through `notification`, in `.github/hooks/*.json`, with `preToolUse` able to deny and `agentStop` able to block. The whole of `docs/10-MECHANICAL-ENFORCEMENT.md` was rewritten; Pitfall 19 above is a retraction. Every correction-capture / build-gate / lint-fix / doc-freshness pattern now has a hook mapping.
- **"Cross-agent invocation has no allowlist."** In VS Code the `agents:` property on a custom agent *is* the allowlist for what it may invoke as a subagent (Pitfall 9). The cloud agent still has none. The framework's *design* choice — specialists return `recommended_next_agent` rather than chaining — still stands, because a flat tree is auditable and a deep one is not; but it is a choice on VS Code now, not a platform gap.
- **"Prompt files are the Copilot equivalent of skills."** They are not. Agent Skills (`.github/skills/<name>/SKILL.md`) are the cross-surface primitive — cloud agent, code review, CLI, VS Code and JetBrains — and are the Copilot equivalent of Claude Code skills. Prompt files are an IDE-only convenience; the VS Code docs' own advice for a prompt file an agent won't use is "convert it to an agent skill". Chapters 2, 3, 5 and 6 were corrected.
- **"Code review reads only the first 4,000 characters."** That sentence is gone from the current docs. The documented limits are now "about 1,000 lines" per instruction file and "no longer than 2 pages" for repository instructions (Pitfall 2). Front-loading stays good practice; it is no longer a cap.

Also changed, quietly: chat modes are retired (Pitfall 7); the agentic `copilot` CLI replaced `gh copilot suggest` as the headless surface (Chapter 6); VS Code reads `.claude/agents/`, `.claude/rules/` and `.claude/settings.json` hooks by default.

Right answer: every framework claim about the platform carries a *verified-on* date. The REFINEMENT prompt now includes a "platform drift" pass: re-read the custom-agents reference, the Agent Skills page, the hooks reference and the Copilot CLI page, and diff them against Chapters 3, 5, 6 and 10. Treat a claim older than a quarter as `documented-unverified` (Chapter 11) until re-checked. Where a Copilot equivalent of something is *not* on the current docs, say "not documented" rather than guessing.

Pitfall 21: An instruction file is a claim, not evidence

An instruction file documented the wrong HTTP verb for a route. Three agents trusted it, shipped a client that called the wrong verb, and every user of that flow saw a generic "check your connection" message for a week. The fourth agent read the route's `export` line.

Instruction files are written by people and agents at a point in time. They rot like any other document — faster, because they are short and confident, and because `applyTo:` auto-loading puts them in front of the agent with the authority of a system prompt.

Right answer: an instruction file is `documented-unverified` until you have looked at the thing it describes. Before *relying* on a documented fact to build something, check it at the source (the route, the schema, the config). When an instruction file is found wrong, fix it in the same turn — never leave a known-false rule standing because "it's not my task".

Pitfall 22: Deferred work that lives only in prose vanishes

"We'll do the retry logic in a follow-up" was said at the end of a session. No follow-up happened. Two weeks later the missing retry was reported as a bug, investigated from scratch, and fixed — with none of the context the first session had.

Right answer: a turn that names deferred work may not end until that work is appended to `docs/<AREA>_BACKLOG.md` with what / why / effort / revisit-when (Chapter 11). The backlog id is checked against the remote and open PRs first — two concurrent sessions took the same id on the same day. Verbal follow-ups are not follow-ups.

Pitfall 23: A production push that does not freshen the docs strands the next agent

A feature went to production on Tuesday. Its orientation map still described it as staging-only behind a flag. On Thursday a fresh agent "enabled" it — and re-opened a migration that had already been applied.

Right answer: every production push updates the full doc set *in the same turn*: `docs/CHANGELOG.md` (the anchor), the affected orientation maps, the affected instruction files, and `PROJECT.md` §3 if the production *state* changed. Documentation discipline missed this about one push in five; it is now an `agentStop` hook (doc-freshness gate, Chapter 10 Pattern 5). A future agent sees only what is in git.

Pitfall 24: Correction-capture regexes false-fire on benign phrases — and the cost is trust

The correction-capture hook (Chapter 10, Pattern 2) fired on *"You already have access to it"* — a perfectly polite sentence — because its regex matched `you already`. It also fired on a test plan that *quoted* a correction phrase, and on a framework doc that *explained* the hook.

Right answer: three guards, all now in the template. (1) Anchor the frustration verbs to what follows them (`you already (did|changed|broke)`), never bare. (2) Strip code fences, inline code and heredoc bodies before matching — quoted text is data, not a correction. (3) Keep the loop guard (`stop_hook_active`, which `agentStop` receives on its stdin payload) so a false fire costs one reply ("not a correction"), never a trapped session. When a false fire does happen, tighten the pattern the same day; a hook that cries wolf gets disabled by the team within a week.

Pitfall 25: A killed check is inconclusive, not failed

The build-gate `agentStop` hook capped the build at five minutes. On a machine also running two other builds, a legitimate four-minute build was killed every time — and reported as *failed*, with a warnings-only tail the agent then tried to "fix". An alarm loop with nothing to fix.

On Copilot there is a second timer to respect: the hook's own `timeoutSec` (default 30), and a hook that exceeds it **fails OPEN by design** — the platform treats it as if it had allowed the stop. So a build-gate has two bad shapes: one that reports a kill as a failure (the alarm loop), and one whose build runs longer than the hook timeout (a gate that is open precisely when the build is slow, with no signal that it opened).

Right answer: distinguish "killed by timeout or signal" from a non-zero exit inside the script. A killed run has produced **no failure evidence**; exit 0 silently and let CI (which builds every PR anyway) be the authority. Only a real non-zero exit blocks the stop. Size the gate's *own* internal cap below the hook's `timeoutSec`, and size both to the slowest honest build on a busy machine, not to the fastest one on an idle machine.

Pitfall 26: Many sessions, one working directory

Three agent sessions shared one checkout. One was launched into a stale, detached snapshot of the tree and "fixed" code that had already been rewritten. Two ran the build at once and corrupted each other's output directory — every route 500'd with an error that read like a code bug and was pure directory collision. Two picked the same migration number.

Right answer: `git fetch` and compare to the remote base *before the first edit* — the tree you were launched in is not trustworthy. Do feature work in an isolated git worktree branched from the fresh-fetched base — use separate git worktrees per session, and symlink the dependency directory rather than duplicating it. Never run a build and a dev server in the same directory at once. Before taking any sequential id (migration number, backlog id, schema version), check the remote *and* other sessions' open PRs. (The Copilot CLI has no `--cwd` flag — run it *from* the worktree.)

Pitfall 27: Never report a negative from a reader you have not verified can see the whole set

"The generator refused these 336 items" turned out to mean "the reader that listed the items was silently capped at 1,000 rows and never showed the generator 25% of them." A confident negative result, wrong, because the *reader* had a ceiling nobody checked.

Most data-access layers cap a bare query (ORMs, REST layers, search APIs — 1,000 is a common default), and an explicit `limit` above the cap often does not raise it. No error, no truncation flag.

Right answer: before reporting "none found", "it refused", or "zero exist", establish the reader's ceiling. Page with a stable order, aggregate in the database, or use a count query. A capped read is worse than a failed one: it looks like a confident answer.

Pitfall 28: Two words for one thing ships bugs

The same concept was called `domain` in the database, `topic` on one component and `chapter` on the page — and `topic` *already meant something else* in a legacy subsystem. A gate took the new meaning, looked it up where the old meaning lived, matched nothing, and an empty match silently widened a filter to the whole dataset. Every user of that gate got off-target results for a day.

Vocabulary drift is not cosmetic. It is how a correct-looking lookup returns the wrong set.

Right answer: one glossary (`docs/ai-context/GLOSSARY.md` — a rule file, not a nicety) naming each concept exactly once, with the DB column, the type field and the UI label that carry it. Adding a fifth name for an existing concept is a defect. Rename only while already editing the file that carries the wrong name, and say so in the commit.

Pitfall 29: A context system is a program — profile it and bisect it

The router, instruction files, skills and hooks you build with this framework are injected into sessions as context, and that cost compounds invisibly: a mature install can front-load thousands of lines into every session and nobody notices until the premium-request bill or the drift does. Worse, misbehavior gets misattributed — when the agent rabbit-holes or fixates, teams blame the model while a stale instruction file or an over-broad skill is doing the steering.

Right answer: treat context like code, with two operational habits. **Bisect:** reproduce the misbehavior on a scratch branch with `.github/copilot-instructions.md` , `.github/instructions/` and the skills temporarily moved aside; if it disappears, the fault is in your context files — find it by restoring halves. **Weigh:** once a quarter, measure what the install injects — which instruction files' `applyTo:` globs matched a typical session, times their size, plus the skills that auto-loaded — and read the org's usage reports for runaway consumers. Make every instruction file earn its context — REFINEMENT check 11 makes this a standing pass. A rule that has never changed an outcome is not free; it is paid for on every session.

Pitfall 30: An unattended job without a verified retire path runs forever

A scheduled pipeline ran for 17 days without a single job completing. The completion write violated a database CHECK constraint, its error return was never read, the job stayed "running", and a reaper re-queued it — so the system burned its full capacity re-doing satisfied work, while every dashboard showed green because runs *were happening*. Nothing a person watched distinguished "ran" from "worked".

Right answer: for any unattended loop (a scheduled workflow, a queue drainer, a standing routine — Chapter 13): the completion write's error is READ, and a failed completion is a loud failure, not a silent retry; attempt caps park a grinding job instead of letting it spin; and the reporting surface states what was *verified done*, never just that the workflow exited green. "The routine ran" is a claim about the scheduler; only the checked completion write is a claim about the work.

Pitfall 31: A scoping field the tool does not recognise fails SILENTLY — and two tools fail in opposite directions

Path-scoped instruction files carry a frontmatter field naming the globs they apply to. Get that field wrong — a typo, a name invented before the platform shipped its own, one copied from a different tool — and nothing errors. The file is still valid. The frontmatter still parses. What changes is invisible, and it is **not the same failure on every tool**:

Tool	Scoping field	Wrong or missing field	What you get
Claude Code	`paths:`	loads **unconditionally**	fails **OPEN** — every file's rules in every session
GitHub Copilot	`applyTo:`	**not applied automatically**	fails **CLOSED** — the guidance never arrives

Same mistake. Opposite damage. On one you drown: a real project's fourteen rule files, ~101k tokens, entered *every* session — a docs edit carrying the payment rules — and the cost was not just tokens, because oversized instruction files are precisely what makes a model start ignoring the instructions that matter. On the other you starve: the rules simply never load, the agent works without them, and the output looks like a model that ignored your standards.

Neither prints a warning. Both look exactly like a working setup.

Right answer: confirm the field name against the tool's current documentation, not against another tool's convention or your own memory (Pitfall 18). Then confirm the *globs resolve*, which the next pitfall covers, because a correct field name pointed at a glob that matches nothing is the same silence wearing different clothes.

Pitfall 32: Glob metacharacters in directory names — the scoped rule that matches nothing

A framework that puts punctuation in directory names hands you globs that are silently invalid. The characters that matter are the ones glob syntax has already claimed:

- `[` **opens a character class.** A directory literally named `[id]` written as `[id]` matches a single character — `i` or `d` — and never the folder.
- `(` **opens a group.** A directory named `(admin)` written as `(admin)` is read as an extglob group and matches nothing at all.

This is not exotic. It is the default routing convention of several mainstream web frameworks (dynamic segments in brackets, route groups in parentheses), and any project using one will write these paths naturally into a rule's globs and never learn they are dead.

Measured on a real repository during exactly this migration: **5 globs** broken by brackets, and **13 more across 7 rule files** broken by parentheses. Every one looked correct in review. The parenthesis case was not even predicted by the person doing the migration — it surfaced only because a check ran before the commit.

Right answer: escape the metacharacter — `\[id\]`, `\(admin\)` — and **verify, do not review.** These patterns cannot be validated by reading them; both failures look like ordinary paths. Ship a checker alongside the rules (`templates/verify-rule-globs.mjs`) that asserts every scoped file matches at least one real tracked file, and run it whenever a glob changes. A rule that matches nothing never loads, and nothing anywhere will tell you.

Two corollaries worth keeping:

- **If a rule and a hook both read these globs, they must agree on one pattern.** A hand-written matcher that treats `[` as a literal and a native matcher that treats it as a class will disagree about the same file, so the checker compares both engines and fails on a disagreement.
- **Where the tool's own documentation does not specify its glob implementation** — as VS Code's does not for `applyTo` (verified 2026-08-25) — a check is not belt-and-braces, it is the only source of truth available.

09 — Runbook: Bootstrap a New Project

> **v1.2.0 additions to this runbook:** after the specialists and instruction files exist, BOOTSTRAP Step 13 generates the project-truth set (`docs/ai-context/PROJECT.md` , `LEARNINGS.md` , `GLOSSARY.md` , the `<slug>-engineering` skill under `.github/skills/` , one backlog) — Chapter 11. Repeatable workflows are now **skills** (`.github/skills/<name>/SKILL.md` , cross-surface); prompt files are IDE-only. Hooks (`.github/hooks/*.json`) are a later hardening phase — Chapter 10. Multi-repo setups — Chapter 12. The short form of Phases 1–7 is `docs/00-QUICKSTART.md` Part 2.

Step-by-step guide for adopting this framework on a real codebase using GitHub Copilot. Plan for ~2-4 hours of focused work.

Prerequisites

- **GitHub Copilot subscription** (Individual / Business / Enterprise) and signed in
- **Copilot extension** installed in your IDE (VS Code / Visual Studio / JetBrains / Eclipse / Xcode)
- **Recent extension version** that supports custom agents (`.github/agents/*.agent.md`)
- **Git** installed; **GitHub CLI** (`gh`) optional for PR creation
- **Read access** to this framework on your disk (`<framework path>` — `~/frameworks/copilot` if you followed the quickstart)
- **2-4 hours** of focused time

Phase 0 — Decide if you need this framework (10 min)

Skip this framework if: - Your project is a solo prototype - Your project has fewer than 4 distinct domain areas - Your team uses Copilot only for inline completions (not Chat or Cloud Agent) - Your project is < 5,000 lines of code

Use this framework if: - Your project has multiple roles - Multiple data systems are in play - Real compliance/security/payments concerns exist - A team uses Copilot daily on the same codebase - You've experienced cascading hallucinations or context drift

Phase 1 — Inventory your project (30 min)

Open your IDE in the project root. Open Copilot Chat.

Paste `prompts/INVENTORY-PROMPT.md` (from this framework). Copilot will: - Scan your codebase structure - Propose a list of specialists based on your domain boundaries - Surface clutter that should be archived - Identify your protected branches - Identify your tech stack and which MCP servers will be useful

You answer: confirm/adjust the proposed specialist list.

Common gotcha: Copilot may propose 10-15 specialists for a medium project. That's too many. Aim for 5-8.

Phase 2 — Bootstrap the framework files (45 min)

In the same Copilot Chat session, paste `prompts/BOOTSTRAP-PROMPT.md` . Reference this framework's path so Copilot can read templates:

```
The framework lives at <framework path>.
Read templates from there. Customize for this project: <project-name>.
```

Copilot will: 1. Create `.github/agents/<project>-orchestrator.agent.md` from the orchestrator template 2. Create one `.github/agents/<specialist>.agent.md` per specialist 3. Create `.github/instructions/<domain>.instructions.md` files (with `applyTo:` globs) 4. Create the starter skills `.github/skills/investigate-bug/SKILL.md` and `.github/skills/build-feature/SKILL.md` , and install the shipped `commit-push-pr` , `correction-capture` , `verify-build` skills (cross-surface; IDE-only prompt-file twins are optional) 5. Create `docs/ai-context/HANDOFF_SCHEMA.md` 6. Create `docs/ai-context/INDEX.md` with task → docs map 7. Create `docs/ai-context/ORCHESTRATION_SPOONFEEDER.md` 8. Create skeleton `docs/ai-context/<area>-experience.md` files 9. Create `.github/copilot-instructions.md` (the router) 10. Create `docs/_archive/README.md` 11. Update `.gitignore` 12. Create the project-truth set (BOOTSTRAP Step 13): `docs/ai-context/PROJECT.md` , `LEARNINGS.md` , `GLOSSARY.md` , `.github/skills/<project-slug>-engineering/SKILL.md` , one `docs/<AREA>_BACKLOG.md`

Verify each file before saving.

Phase 3 — Add your first instructions (30 min)

The hardest part of this framework is writing useful instruction files. They take real domain knowledge.

In the Copilot Chat session, ask:

```
Based on the codebase scan, propose 3–5 instruction files for .github/instructions/.
For each, list:
  – applyTo: glob list (verify the globs match real files in this project)
  – 2–3 hard rules with WHY + HOW TO APPLY
  – excludeAgent: if relevant

Do not propose any rule that's just style preference — only invariants where
breaking them caused or could cause a real production issue. Cite file:line
where the gotcha currently lives.

Wait for my approval before creating any files.
```

Common instruction files to start with: - `backend-api.instructions.md` (or your equivalent server-side concern) - `frontend-ui.instructions.md` (or `mobile-ui.instructions.md`) - `database.instructions.md` - `auth-security.instructions.md` - `testing.instructions.md`

Aim for **3-5 rules per file** at first.

Phase 4 — Check the 1-2 starter skills (20 min)

Most projects benefit from at least these (BOOTSTRAP Step 8 creates them; verify or ask for them here): - `.github/skills/investigate-bug/SKILL.md` - `.github/skills/build-feature/SKILL.md`

Ask Copilot:

```
Create .github/skills/investigate-bug/SKILL.md and
.github/skills/build-feature/SKILL.md using the skill structure in
docs/05-INSTRUCTIONS-AND-PROMPTS.md (frontmatter `name` + `description`;
the workflow body follows templates/prompt.md.template), customized for
this project's tech stack and roles.

These should be invokable via /investigate-bug and /build-feature on every
surface (IDE chat, cloud agent, CLI).
```

You can add more skills later (qa-flow, compliance-review, audit-pipeline, context-refactor). A prompt file (`.github/prompts/*.prompt.md`) is only for a workflow you start by hand in the IDE and that needs `${input:…}` prompting.

Phase 5 — Verify (15 min)

Run these checks:

```
# 1. Files in expected locations
ls .github/agents/
ls .github/instructions/
ls .github/skills/
ls .github/prompts/      # only if you created IDE-only prompt files
ls docs/ai-context/

# 2. Doc-link sweep — find broken refs
grep -rohE "(docs|\.github)/[A-Za-z0-9_./-]+\.md" .github/copilot-instructions.md docs/ .github/agents/ .github/instructions/ .github/skills/
        .github/prompts/ 2>/dev/null | sort -u | while read p; do [ -f "$p" ] || echo "BROKEN: $p"; done

# 3. Build still passes (whatever your build command is)
npm run build  # or: pytest, go build, cargo build, etc.
```

In your IDE: 4. **Reload the IDE** so Copilot picks up the new `.github/` files. 5. Open Copilot Chat → click the agent dropdown → confirm your specialists appear in the list. 6. Type `/` in Chat → confirm your skills (and any prompt files) appear in the picker.

Fix any breakage before moving on. Common issues: - Agent file YAML invalid (it won't appear in the dropdown) - `applyTo:` glob points to a path that doesn't match anything (instruction won't load) - Markdown link in `.github/copilot-instructions.md` to a file that doesn't exist (typo)

Phase 6 — Run a real task through the orchestrator (30 min)

Pick a small real task — ideally a bug fix or a small feature.

Open Copilot Chat. Select your `<project>-orchestrator` agent from the agent dropdown (or type `@<project>-orchestrator` at the start of your message).

Verify the active agent shown in Chat is your orchestrator.

Give it the real task. Watch carefully: - Does the orchestrator emit the outbound `handoff:` YAML block when it proposes to delegate? - When you accept the delegation (or invoke the specialist), does the specialist return the inbound `return:` YAML block? - Does the orchestrator catch any rejected_claims correctly? - Does verification (tests, build) actually run?

If anything is off → tighten the relevant agent's body. Common fixes: - Orchestrator skipping the YAML block → tighten its "outbound discipline" section - Specialist accepting vague delegations → tighten its "incoming handoff validation" section - Specialist not running tests → add explicit "tests_run MUST be non-empty" to its Definition of Done

Phase 7 — Commit, open a PR, and document for your team (20 min)

Commit on the setup branch and open a PR (`.github-pre-bootstrap-backup/` stays gitignored):

```
git add .github/ docs/ .gitignore
git commit -m "chore: bootstrap Copilot orchestration framework"
git push -u origin setup/copilot-orchestration
gh pr create --base <your-base-branch> --title "Bootstrap Copilot orchestration"
```

Add to your project's existing onboarding docs: - "We use GitHub Copilot with a custom orchestration setup. See `docs/ai-context/ORCHESTRATION_SPOONFEEDER.md`." - "Always pick the right invocation mode — see `docs/06-INVOCATION-MODES.md` (or the spoonfeeder's invocation modes section)." - "Don't push to main without project-owner approval."

Optional: announce in your team chat with a 30-second demo of `@<project>-orchestrator` handling a multi-step task.

Phase 8 — Add hardening as needed (optional, after 2 weeks)

After running real tasks for a couple weeks, decide if you need:

Tighter `tools:` allowlists

If a specialist is doing things outside its domain (e.g. running terminal commands when it shouldn't), tighten its `tools:` array.

Organization-level custom instructions (Business / Enterprise)

If you have multiple repos using this framework, define shared cross-repo conventions at the organization level (UI at github.com org settings + `.github-private/agents/` for shared agents).

CI check for handoff schema compliance

If specialists keep emitting malformed return blocks, add a CI script that grep's PRs for the schema and warns when missing. (Custom CI, not a Copilot feature.)

`excludeAgent: "code-review"` on noisy instructions

If code review is over-applying instructions (e.g. flagging style preferences), add `excludeAgent: "code-review"` to those instruction files.

Phase 9 — Quarterly maintenance

Every 3 months, dedicate a session to: 1. Run `prompts/REFINEMENT-PROMPT.md` (and the `/context-refactor` skill, or have the `context-librarian` specialist do a sweep) 2. Move stale dated reports to `docs/_archive/<YYYY-MM>/` 3. Reconcile any instruction conflicts 4. Update orientation maps for areas where reality has drifted 5. Verify all agents still appear in the IDE Chat dropdown after dependency / IDE updates

Common time-sinks to avoid

- **Don't try to make every instruction file perfect on day 1.** Ship 5; add 1 per month as production teaches you.
- **Don't over-specialize.** 6-8 specialists is plenty.
- **Don't write canonical docs from scratch in this framework.** Use what you already have.
- **Don't manually merge agent files when adding a rule.** Always edit the instruction file.
- **Don't forget to reload the IDE after `.github/` changes.** New agents won't appear until reload.

When to stop and ask

- If specialists don't appear in the IDE Chat dropdown → YAML frontmatter syntax error
- If instruction files don't auto-load → check `applyTo:` glob matches the file you're editing
- If specialist outputs don't include the return YAML block → the specialist's body needs the "Return schema" section
- If the orchestrator keeps delegating in a loop → check `failure_condition` is articulated
- If a specialist refuses a delegation → check the orchestrator's outbound block has all required fields

End state

You have: - A `<project>-orchestrator` and 4-10 specialists in `.github/agents/` - 3-5 path-globbed instruction files in `.github/instructions/` - 2-4 skills (slash commands on every surface) in `.github/skills/`, plus the `<project-slug>-engineering` playbook skill; optional IDE-only prompt files in `.github/prompts/` - Three-tier docs (orientation maps + canonical refs + archive) - A team that knows how to pick invocation modes - Auditable handoffs for every cross-domain task

Time invested: 2-4 hours setup + ~30 min/quarter maintenance.

Time saved: every cascading-hallucination incident you DIDN'T have. Every cross-domain task that got the right specialists without you thinking about it. Every new engineer who could navigate the codebase without 2 weeks of pairing.

10 — Mechanical Enforcement (hooks, skills, and what still needs discipline)

> **Rewritten for v1.2.0.** The v1.1 version of this chapter opened with *"GitHub Copilot does not expose programmable lifecycle hooks."* That was true when written and is false now: Copilot has hooks on the cloud agent, the Copilot CLI and VS Code. Every platform fact below was verified against the official hooks reference and the VS Code hooks page on **2026-08-22**. Re-verify quarterly (Pitfall 20 — the platform moves under your conventions).

This chapter is **optional**. Skip it until you have concrete evidence that documentation discipline is being skipped under real-world pressure (the same triggers as the Claude edition: the same correction repeated across sessions, bugs shipping despite a rule that forbade them, builds skipped on commits, a production push with stale docs behind it). Then come back.

What Copilot actually offers (verified 2026-08-22)

Surface	Mechanism	Where
Declarative rule loading	`.github/instructions/*.instructions.md` with `applyTo:` globs — auto-loaded when a matching file is in play	All surfaces
Lifecycle hooks	`.github/hooks/*.json` — events `sessionStart`, `sessionEnd`, `userPromptSubmitted`, `preToolUse`, `postToolUse`, `postToolUseFailure`, `agentStop`, `subagentStart`, `subagentStop`, `errorOccurred`, `preCompact`, `permissionRequest`, `notification`	Cloud agent (reads only `.github/hooks/*.json`; `bash` / `command` fields only) · Copilot CLI (all events; also `~/.copilot/hooks/`) · VS Code (reads `.github/hooks/*.json` **and** a Claude-style `.claude/settings.json`)
Skills	`.github/skills/<name>/SKILL.md` — invoked as `/name` or auto-loaded by relevance	Cloud agent, code review, CLI, VS Code, JetBrains
Prompt files	`.github/prompts/*.prompt.md`	IDEs only — not the cloud agent, not the CLI

The hook contract (the part you must get right)

```
{
  "version": 1,
  "hooks": {
    "preToolUse":  [ { "type": "command", "bash": "node .github/scripts/pre-tool.mjs",  "timeoutSec": 10 } ],
    "postToolUse": [ { "type": "command", "bash": "node .github/scripts/post-tool.mjs", "timeoutSec": 30 } ],
    "agentStop":   [ { "type": "command", "bash": "node .github/scripts/stop-gate.mjs", "timeoutSec": 600 } ]
  }
}
```

- **Input:** JSON on stdin. `agentStop` receives `session_id`, `cwd`, `transcript_path`, `stop_reason`, `stop_hook_active`. `preToolUse` / `postToolUse` receive `tool_name`, `tool_input` (and `tool_result` after). `userPromptSubmitted` receives `prompt`. Field names also arrive camelCase on some surfaces (`sessionId`, `toolName`, `transcriptPath`) — read both.
- **Output:** JSON on **stdout**. `preToolUse` → `{"permissionDecision": "allow|deny|ask", "permissionDecisionReason": "…"}`; `agentStop` → `{"decision": "block", "reason": "…"}` to force another turn; `postToolUse` → `{"additionalContext": "…"}`.
- **Exit codes:** `0` = parse stdout. For `preToolUse`, `2` and any other non-zero = **deny** (fail-closed). For other events a non-zero exit is a warning; `stderr` is shown to the user and the run continues.
- **Timeouts always fail OPEN**, whatever the event. A gate that needs to *block* must finish inside `timeoutSec` or it silently allows. Size the timeout to the slowest honest run and, inside the script, treat your own internal timeout as *inconclusive* (Rule 9 below).
- **Loop guard:** `agentStop` passes `stop_hook_active: true` on the forced continuation. Exit `{"decision":"allow"}` when you see it, or you trap the agent.

This contract differs from Claude Code's in two ways that matter if you run both editions: the reminder goes on **stdout as JSON** (Claude: stderr as text), and `preToolUse` is **fail-closed** (Claude: fail-silent). Do not copy a hook script between the two without re-reading this table.

Translation table – the five patterns on Copilot's surface

Pattern	Copilot mechanism	Quality
1 Rule-surfacing (load matching rules before an edit)	**Native.** `applyTo:` on instruction files. No script.	Strict
2 Correction-capture (a correction must become an instruction-file patch)	`userPromptSubmitted` hook detects the correction signal and writes a flag file; `agentStop` hook blocks the stop while the flag exists	Native (v1.2) – previously a manual `/correction-capture` prompt; keep the prompt for IDEs without hooks
3 Build-gate (no stop with build-relevant files dirty and a failing build)	`agentStop` hook runs the build; `{"decision":"block"}` on a real failure; timeout = inconclusive	Native (v1.2) – plus Definition-of-Done discipline as before
4 Lint-fix after edit	`postToolUse` hook on edit tools, or editor format-on-save / pre-commit	Native (v1.2); IDE config still preferred
5 Doc-freshness gate (a production push must be followed by a changelog edit in the same session)	`postToolUse` hook records the last push and the last changelog edit in a state file; `agentStop` blocks when push > changelog	Native (v1.2)
`/commit-push-pr` daily workflow	`.github/skills/commit-push-pr/SKILL.md` (works on every surface; the v1.1 prompt-file version is IDE-only)	Native

Templates for all of these ship at `templates/hooks/` (Copilot JSON contract) and `templates/skills/`. Each script is stack-agnostic with `<UPPERCASE>` placeholders for the build command, changelog path and protected branch.

Why the flag-file design instead of parsing the transcript

Claude Code's Stop hooks parse the session transcript (a documented JSONL format). Copilot's `agentStop` also hands you a `transcript_path`, but its format is not documented in the hooks reference and may differ per surface. The templates therefore never parse it: `userPromptSubmitted` and `postToolUse` see the events directly and record what the `agentStop` gate needs in a small state file under `.github/hooks/.state/` (gitignored). Same behaviour on every surface, no dependency on an undocumented format.

Pattern 1 – Rule surfacing (native)

Unchanged from v1.0. `applyTo:` globs auto-load the instruction body when a matching file is being edited or reviewed. Discipline that still matters: keep each file short (current guidance: ~1,000 lines max; two pages for the repo-wide file), make the globs actually match (verify with a `find`), and audit `applyTo:` overlap quarterly.

Pattern 2 – Correction-capture (two hooks + a flag)

1. `userPromptSubmitted` → `correction-detect.mjs`: strips code fences and inline code from the prompt, runs the anchored correction regexes (Rule 11), and on a match writes `.github/hooks/.state/correction-pending`.
2. `agentStop` → `stop-gate.mjs`: if the flag exists and `stop_hook_active` is false, deletes the flag and returns `{"decision":"block","reason":"<the §9 reminder: draft a one-line patch to the right .github/instructions/<file>.instructions.md and ask for approval – never 'I'll remember'>"}`.

The `/correction-capture` skill remains for surfaces where you have not installed hooks.

Pattern 3 – Build-gate (`agentStop`)

`stop-gate.mjs` checks `git status --porcelain` for dirty build-relevant files; if any, runs `<BUILD_COMMAND>` with its own internal cap **below** the hook's `timeoutSec`. Real non-zero exit → `{"decision":"block","reason":"<build tail>"}`. Internal cap hit → allow (inconclusive, CI builds every PR). Loop-guarded.

Pattern 4 – Lint-fix (`postToolUse`)

`lint-fix.mjs` runs the project's auto-fix linter on the file in `tool_input` when `tool_name` is an edit tool. Always exits 0 and outputs nothing – a lint problem must never block.

Pattern 5 – Doc-freshness gate (`postToolUse` + `agentStop`)

`doc-freshness-track.mjs` (postToolUse) records two timestamps in the state file: the last shell command that was a *real* production push (its own top-level command segment starting with `git push` / `gh pr merge` and naming `<PROTECTED_BRANCH>`; heredoc bodies and quoted text never count – Rule 10) and the last edit to `<CHANGELOG_PATH>`. `stop-gate.mjs` blocks when the push is newer than the changelog edit, with the full doc-update checklist (changelog → affected orientation maps → affected instruction files → `PROJECT.md` §3 if production state changed – Chapter 11).

Design rules for any Copilot hook

1. **Decide fail-open vs fail-closed per event, on purpose.** `preToolUse` is fail-closed by the platform (a crashing hook denies every edit). Wrap `main()` in try/catch and emit `{"permissionDecision":"allow"}` on unexpected errors unless the hook is a security gate.
2. **Scope narrowly.** Filter on `tool_name` / file path first; exit fast.
3. **Cap output.** `additionalContext` and `reason` land in the model's context.
4. **Loop guard** every `agentStop` hook on `stop_hook_active`.
5. **One `agentStop` script, ordered checks inside it.** Copilot runs the hooks array in order but the first `block` wins; keeping correction → build → doc-freshness inside one script makes the order explicit and the state file reads cheap.
6. **stdout is the channel, and it must be valid JSON.** Debug to stderr.
7. **Commit `.github/hooks/*.json`.** The cloud agent reads only that location; a hook in `~/.copilot/hooks/` enforces nothing for teammates or CI.
8. **Restart to load.** Hook files are read at session start on the CLI and VS Code; verify in a fresh session, not the one that installed them.
9. **A killed check is inconclusive, not failed.** Your script's internal build cap must be lower than the hook `timeoutSec` (which fails open anyway), and a cap hit must `allow`, never `block`.
10. **Quoted text is data.** Strip fences, inline code and heredoc bodies before pattern-matching; match shell commands per top-level segment with the verb anchored at the start.
11. **Anchor correction regexes.** Bare `you already` matched "you already have access". A false positive costs more trust than a miss.
12. **A push in another repository is not this repo's push.** `doc-freshness-track` tracks where the shell is per command segment (`cd`, subshell `cd`, `git -C`, `gh --repo`) and records a push only when its effective directory is inside the session's `cwd`; an unknowable directory (`cd $VAR`) never counts. Releasing sibling repos from scratchpad clones fired the Claude edition's gate twice before this guard existed.
13. **Content beats ordering.** Teams that write the changelog entry FIRST (changelog → integration branch → promote) ship it inside the promote merge, so no changelog edit follows the push. The doc-freshness gate also accepts a same-day release heading in the changelog (read from `origin/<PROTECTED_BRANCH>` or the working tree); customise `RELEASE_HEADING_RE` to your heading shape, and make sure another environment's heading or a stale date cannot match.

What still needs discipline (no hook can do it)

- The handoff schema being present and evidence-bound on every delegation.
- A specialist refusing a vague delegation.
- The orchestrator actually validating `return:` blocks (`tests_run` includes the build; deferred work has a backlog path; `contracts_changed` is backward-compatible).
- Reading an instruction file *before planning*, not only when `applyTo:` fires on an edit.

These live in the agent bodies and the Definition of Done, exactly as in v1.0.

Verification

1. **Hook files load** — start a fresh CLI session in the repo; a `sessionStart` hook that prints one line of `additionalContext` proves the file was read.
2. **Correction-capture** — type a real correction; attempt to stop; expect the block with the reminder; reply with the patch; second stop succeeds (loop guard).
3. **Build-gate** — break the build; attempt to stop; expect the block with the build tail; fix; stop succeeds.
4. **Doc-freshness** — run a (dry) `git push origin <PROTECTED_BRANCH>` in the session; attempt to stop; expect the block; edit the changelog; stop succeeds. Then paste a doc that *quotes* the push command and confirm no block.
5. **Negative test** — a docs-only turn must produce no output from any hook.
6. **Cloud agent** — assign a trivial issue; confirm in the run log that `.github/hooks/*.json` was loaded.

If any of these fail, treat it as P0 — a hook that silently allows is worse than no hook.

Cross-links

- `docs/01-PRINCIPLES.md` § Principle 5 — runtime-enforced vs documented (the line moved in v1.2)
- `docs/05-INSTRUCTIONS-AND-PROMPTS.md` — `applyTo:` mechanics; skills vs prompt files
- `docs/08-COMMON-PITFALLS.md` § Pitfall 19 (hooks exist — retraction), § Pitfall 20 (platform drift)
- `docs/11-PROJECT-TRUTH-AND-LEARNINGS.md` — the rule Pattern 5 enforces
- `templates/hooks/` · `templates/skills/`
- Official: *GitHub Copilot hooks reference*, *VS Code → Hooks*, *About agent skills* (verified 2026-08-22)

11 — Project Truth, Learnings, and the Evidence Ladder

> Added in v1.2.0. Distilled from three further months of running the framework on a production codebase with many parallel agent sessions. Everything here is tech-stack and domain agnostic, and nothing in it depends on a Copilot surface — it applies to the cloud agent, IDE chat, and the CLI alike.

The v1.0 framework answered *"who does the work and how do they talk to each other?"* (agents, handoff schema, instruction files). Three months of real use exposed the next failure class: **a fresh agent with no transcript knows only what is in git — and git mostly recorded *code*, not *truth*.**

Symptoms, all observed more than once:

- An agent built a feature that already existed, because nothing said "this exists, it is just hidden behind a flag".
- An agent treated a staging-only feature as live in production, because the orientation map described the feature but not *where it was deployed*.
- A "we'll do that later" spoken at the end of a session vanished, and was re-discovered as a bug two weeks on.
- An instruction file documented the wrong HTTP verb for a route. Three agents trusted it. The fourth read the route.

The fix is three more documents, one playbook, and one enforced habit. This chapter specifies them.

The three knowledge stores (and what goes where)

Store	File	Answers	Written by	Loaded
Project truth	`docs/ai-context/PROJECT.md`	*What is true right now?* Environments, what is live where, sources of truth per domain, verified commands, definitions of done	Humans + agents, on every state change	Read at the start of every substantial task
Learnings	`docs/ai-context/LEARNINGS.md`	*What did we decide, what failed, what keeps going wrong, how has the owner corrected us?*	Agents, same turn a lesson emerges	Skim §D (corrections) before the first edit
Backlogs	`docs/<AREA>_BACKLOG.md`	*What did we defer, and when should we come back?*	Agents, before any turn that names deferred work ends	On "what's pending on X?"

These sit alongside the stores v1.0 already had: `.github/copilot-instructions.md` (router — or `AGENTS.md` if the project wants the router read by more than one AI tool), `.github/instructions/*.instructions.md` (path-scoped hard rules, `applyTo:`), `docs/ai-context/<area>.md` (orientation), `docs/<UPPERCASE>.md` (canonical reference), and any IDE- or CLI-local memory the tool keeps (not visible to teammates).

One home per fact. Every other place links to it. When the same rule turns up in two files, consolidate and leave a pointer. The single most corrosive doc smell is two copies that drift.

Where a new piece of knowledge goes

Kind of knowledge	Home
Universal behaviour + routing	`.github/copilot-instructions.md` (keep under ~200 lines)
Current state — what is live where, flags, applied migrations	`PROJECT.md` §3 + `docs/CHANGELOG.md`
A decision and its rationale	`LEARNINGS.md` §A
An approach that failed or proved unsafe	`LEARNINGS.md` §B
A recurring bug: symptom → first thing to check	`LEARNINGS.md` §C
A correction the owner had to give more than once	`LEARNINGS.md` §D **and** an instruction file if path-scoped
A hard rule scoped to file paths	`.github/instructions/<domain>.instructions.md` (`applyTo:`)
Per-area orientation (50–150 lines)	`docs/ai-context/<area>.md`
Deferred work	`docs/<AREA>_BACKLOG.md`
A multi-step repeatable procedure	`.github/skills/<name>/SKILL.md`
Machine-private session pointers	Any IDE- or CLI-local memory the tool keeps — **not** visible to teammates or other agents; anything durable must *also* land in one of the above

Don't persist: session-specific context, one-off data values, anything derivable from code or git in seconds, generic engineering advice.

`PROJECT.md` — the freshness contract

Template: `templates/PROJECT.md.template`. The sections that earn their keep:

1. **Product + roles** – one paragraph and a table of roles with their permission boundaries.
2. **Environments, branches, deploy** – and, critically, **what the deploy pipeline does NOT do** (migrations, cache purges, config pushes). "Migrations don't auto-apply" cost a real outage.
3. **Current state – what is live where.** A table: feature → prod / staging / flag / date. This is the single biggest fresh-agent trap: most recent work is usually staging-only and flag-gated.
4. **Repository map** – verified, date-stamped.
5. **Sources of truth by domain** – the canonical helper / table / module for each concept, so an agent reuses instead of re-implementing.
6. **Verified commands** – copied from the real build/test config, with the date.
7. **Definitions of done per change type.**
8. **Known sharp edges + unverified items.**

The contract: every fact is date-stamped, and the header names the commit it was verified against. If today is much later than the stamp, trust `docs/CHANGELOG.md` over the state table, *then fix the table*. A state claim without a date becomes a lie the day after it stops being true.

`LEARNINGS.md` – decisions, failures, corrections

Template: `templates/LEARNINGS.md.template`. Six sections:

- **A. Decision records** – what was decided, by whom, why, dated.
- **B. Failed or unsafe approaches** – so nobody re-tries them. Each entry: what was tried, what broke, what replaced it.
- **C. Recurring bug patterns** – symptom → first thing to check. The cheapest section to write and the one that saves the most hours.
- **D. Agent behaviour corrections** – written as *"Instead of: … / Do: …"* pairs. These came from real corrections; treat them as standing instructions.
- **E. Working with the project owner** – the owner's preferred formats and approval gates. Examples that generalise: *multi-item status = a table, not prose*; *open decisions = one numbered list, resolved one at a time, never drop an item*; *approval is per-incident and current-turn, never carried forward from an earlier session*.
- **F. Documentation maintenance protocol** – the "where does knowledge go" table above, plus: date-stamp state claims; prune on contact (a verified-stale entry is fixed or deleted in the same turn, never left); unresolved conflicts get a ⚠ CONFLICT flag for the owner, never a silent pick.

Where a lesson is already a hard rule, `LEARNINGS.md` carries one line and a pointer – never a copy.

Backlogs – deferred work must be written, not spoken

If an end-of-turn summary names *any* "we'll do this later", follow-up PR, known gap, or deferred phase, the agent **must** append it to `docs/<AREA>_BACKLOG.md` before the turn ends. Template: `templates/BACKLOG.md.template`. Each entry needs:

```
### <Short title>
- **Why.** (one sentence on user value or technical reason)
- **Effort.** (days/weeks)
- **Where.** (file paths or component names if known)
- **Revisit when.** (the condition or signal to look for)
```

When done, prepend `✓ DONE – PR #N (date)` to the title and leave the body in place for context.

Two corollaries learned the hard way:

- **A backlog card is a claim, not evidence.** "Done" on a card proves nothing about production; verify state in `PROJECT.md` §3 or the deployed system.
- **Backlog ids collide across concurrent sessions.** Before taking the next `AREA-N` id (or the next migration number), check the remote *and* other open PRs. Two sessions took the same number on the same day, twice.

Every production push freshens the docs – same turn, no exceptions

The moment a change lands on the production branch, a future agent sees only what is in git. So a production push must be paired, **in the same turn**, with updates to the full doc set for what shipped:

1. `docs/CHANGELOG.md` – the mandatory anchor (a release entry).
2. The affected `docs/ai-context/<area>.md` orientation map(s).
3. Any affected `.github/instructions/*.instructions.md` hard rules.
4. `PROJECT.md` §3 if the production *state* changed (new live feature, flag flip, migration applied).

Documentation discipline alone skipped this under deadline pressure roughly one push in five. It is now an `agentStop` hook (Chapter 10, Pattern 5 – `docs/10-MECHANICAL-ENFORCEMENT.md`) which refuses to end a turn in which a production push happened and `docs/CHANGELOG.md` was not touched afterwards. Teams without hooks encode the same list in the orchestrator's Definition of Done and in the `/commit-push-pr`-style prompt or skill.

The engineering playbook – six gates and the evidence ladder

Template: `templates/engineering-playbook-skill.md.template`, installed as an Agent Skill at `.github/skills/<project-slug>-engineering/SKILL.md`. A project-wide skill that every substantial task runs through; the per-task-type skills (investigate-bug, build-feature) encode the checklists, this one encodes

the gates they share. **If a gate conflicts with a hard rule, the rule wins.**

Gate	What it forces
1 Scope	Restate the user-visible outcome; classify (bug / extension / pivot / data correction / migration / ops / docs); name affected roles and blast radius; say what is *out* of scope
2 Evidence	Read the minimum routed context; **audit what exists before building** (already supported / supported-but-broken / partial / not supported / not needed); trace the real read/write path; tag every claim with a confidence class
3 Design challenge	Smallest viable change + one rejected alternative; find the existing pattern to mirror; state the failure condition; run the standing impact list (permissions, sensitive data, every client, accessibility, rollout compatibility, flag gating, money paths)
4 Implement	Narrow diffs; new user-visible behaviour lands flag-gated default-off; tests at the right level; code + migrations + scripts + docs in the same change
5 Verify	The evidence ladder below – a green build is evidence, not proof
6 Report	Files, instruction files read, checks run **with actual results**, risks and what was *not* verified (with its confidence class), docs updated, follow-ups written to a backlog

The evidence-confidence taxonomy

Mandatory in plans and reports. Nothing below `verified-*` may silently become a premise.

Class	Meaning
`verified-code` / `verified-schema` / `verified-test` / `verified-git`	You looked at the thing itself
`documented-unverified`	A doc says so – including *this project's own instruction files* – and you didn't check
`historical`	True at a past date; may be stale
`unknown`	Needs confirmation

The proof ladder (in order of authority)

1. **The user-visible flow, exercised** – drive the real screen, capture the result. UI claims need UI proof, never code-reading. Hard-to-reach states are *created* (test accounts, seeds), not skipped.
2. **The data, landed** – query for the rows the change was supposed to produce. Proxy signals ("queued", "in scope", "deployed", "nothing to do") prove nothing.
3. **The third party, confirmed** – if an external service is involved, check its dashboard or API, not your code's return value.
4. **Regression edges** – adjacent tests, permission boundaries, empty/error/loading states, *both* sides of any two-sided flow.

Two rules that belong to the ladder:

- **Verify by reading the row, not by reasoning.** Two confident mechanisms were predicted for one write; both were wrong; a single query settled it in seconds.
- **Never report a negative result from a reader you have not verified can see the whole set.** Default pagination caps (ORMs, REST layers, search APIs) silently truncate. A "none found" from a truncating reader is not evidence – check the reader's ceiling first.

Multi-client parity (when more than one client consumes one backend)

A second client is an unusually good audit of the first, and the source of a whole class of drift. These rules are generic to any web + mobile, or any two consumers of one API, and they are what makes Chapter 12's cross-repo contracts workable:

- **One client is the functional source of truth; the others may differ in presentation only.** What a person can *see and do* may not differ between clients. Layout, density, device affordances may. A deliberate divergence is written down with its reason.
- **Same heading ⇒ same endpoint.** Before building a panel that mirrors another client's panel, open that client's component and find what it actually *fetches*. Two cards with the same title and the same chart fed from two different endpoints showed one user two different answers to one question. Record the panel → source pairing in the orientation map.
- **One brain.** Business logic lives server-side; clients render. If a client needs derived data another client computes locally, extend the API rather than porting the computation.
- **Changing a contract? Grep every consumer in the same turn.** Clients pin response shapes; a renamed field fails silently at runtime (fetch succeeds, screen is empty). Keep a **mirror registry** – a table of code deliberately duplicated across clients (palette, label mappers, status derivations) – and change both sides in one PR.
- **Never ship a client that hard-depends on a brand-new server route.** Clients and servers deploy on different clocks, and an installed client can be older *or newer* than the server. Probe or degrade, and say so on screen when degraded.

Smaller lessons that earned a line

- **Search before building.** Grep for the feature first. An existing implementation was rebuilt because nothing advertised it — and a "hide until ready" guard had silently removed it for a week. Guards that hide features need a visible "hidden because" signal somewhere.
- **Diagnostic scaffolding is removed in the same change as the fix.** An on-screen error line that cracked a bug becomes a permanent leak of raw technical text if left in.
- **Two words for one thing ships bugs.** Ambiguous vocabulary (the same word meaning two things in two subsystems) caused a silent filter miss in production. Keep a glossary in `docs/ai-context/GLOSSARY.md`; adding a fifth name for an existing concept is a defect, not style.
- **Corrections harden into instruction files, never into "I'll remember".** Any memory the IDE or CLI keeps is machine-local and invisible to every other agent and teammate.

Cross-links

- `docs/10-MECHANICAL-ENFORCEMENT.md` — Pattern 5 (doc-freshness gate, `agentStop`) enforces the production-push rule.
- `docs/12-MULTI-REPO-WORKSPACES.md` — the parity rules above applied across repositories.
- `docs/08-COMMON-PITFALLS.md` — Pitfalls 20–28 are the incident reports behind this chapter.
- `templates/PROJECT.md.template`, `templates/LEARNINGS.md.template`, `templates/BACKLOG.md.template`, `templates/GLOSSARY.md.template`, `templates/engineering-playbook-skill.md.template`.

12 — Multi-Repo Workspaces (web + mobile + microservices)

> Added in v1.2.0. Platform facts verified against the official GitHub Copilot docs and the VS Code Copilot docs on 2026-08-22 (custom agents, subagents, agent skills, hooks, Copilot CLI, coding agent, agent-customization overview). Re-verify before relying on a behaviour.

The question this chapter answers

> *"We have one repo per microservice, plus a web repo, a mobile repo, and an API gateway. Does orchestration need agents at every level, or can a top-level orchestrator understand all the services? Should we create a separate workspace repo with a workspace file and the agent files, gitignore the sub-projects, and skip CI at that level?"*

Short answer: **three layers, each optional, each building on the one below.** The per-repo framework is layer 1 and stays mandatory. Shared specialists move to the organisation's `.github` / `.github-private` repo (layer 2) once three or more repos would otherwise carry copies. A workspace repo (layer 3) — a `.code-workspace` file plus the cross-repo orchestrator, service map and contract rules, with the children as gitignored clones and no CI — exists only when tasks *routinely* cross repos. The teammate's instinct is right; what needs correcting is **what lives in the workspace** (the orchestrator and the maps, not the specialists) and **how it delegates**.

```
Layer 3  workspace repo            ← .code-workspace, cross-repo orchestrator, service map, contracts, sync script
Layer 2  org-level agents          ← .github-private/agents/ (Business/Enterprise) — specialists every repo would otherwise copy
Layer 1  each repo (required)      ← copilot-instructions.md, .github/agents, instructions, skills, hooks — chapters 1–11
```

What the platform does across several folders

This is the design constraint, and it is *different* from the Claude Code edition:

Thing	VS Code multi-root workspace	Copilot CLI run in a child	Cloud agent
`copilot-instructions.md` / `AGENTS.md`	Discovered **per workspace folder**	The child's	The child's
`.github/instructions/` (`applyTo:`)	Per folder; globs match within that folder	The child's	The child's
`.github/agents/*.agent.md`	**Every folder's agents are loaded** — identity is the `name` field, so two repos' `backend-api` collide	Only the child's (plus `~/.copilot/agents/`)	The assigned repo's (plus org-level)
`.github/skills/`	Per folder	The child's	The child's
`.github/hooks/*.json`	Per folder (VS Code also reads `.claude/settings.json`)	The child's	Only `.github/hooks/*.json` of the assigned repo
Cross-repo changes in one run	Possible (it is one editor session)	No — run it per repo	**No** — "Copilot cannot make changes across multiple repositories in one run"

Two consequences drive everything below:

1. **A multi-root session sees every child's enforcement layer** (instructions, hooks) — good — **and every child's agents** — which means *name collisions*. If three services each ship a `backend-api` agent, the workspace session has three agents called `backend-api` and which one answers is not something you control.
2. **The cloud agent is per repo, full stop.** A cross-repo change is N cloud-agent tasks, sequenced by the contract protocol, never one.

Also verified, and it corrects a v1.0 claim: **custom agents can invoke custom agents.** In VS Code the `agents:` frontmatter list is an allowlist of subagents (`*` = all) and nested invocation is enabled with `chat.subagents.allowInvocationsFromSubagents` ; GitHub's `agent` tool alias does the same on the cloud agent. A workspace orchestrator → repo orchestrator → repo specialists tree is possible inside one VS Code session.

Layer 1 — every repo keeps its own framework install

Nothing here removes the per-repo install. Each service, the web app and the mobile app keep `.github/copilot-instructions.md` , `.github/agents/<repo>-orchestrator.agent.md` + specialists, `.github/instructions/` , `.github/skills/` , `.github/hooks/` and `docs/ai-context/` . Reasons:

- Most tasks are single-repo. A service team opening their repo must get the full experience with zero knowledge that a workspace exists.
- The cloud agent and the CLI see only the repo they run in — the gates must live with the code.
- The child's orchestrator is what the workspace delegates to.

Naming rule for repos that will join a workspace: the orchestrator is `<repo>-orchestrator` (unique by construction). Specialists keep generic names (`backend-api`) *inside* the repo; the workspace never invokes them directly (see mechanisms below), so collisions are avoided by design rather than by renaming every specialist in every repo.

Layer 2 — shared specialists live at the organisation level

Twelve services will share `backend-api`, `database`, `observability`, `release-devops`, `security-privacy` almost verbatim. Copying them into twelve repos guarantees drift. Copilot's own mechanism is **organisation-level custom agents** in the `/agents/` directory of the org's `.github` or `.github-private` repository (Business / Enterprise; precedence by file name is repo > org > enterprise, so a repo can still override one). Put the generic specialists and the shared skills there; keep in each repo the router, the repo-specific instruction files, the orchestrator, `PROJECT.md` / `LEARNINGS.md` /backlogs, and any specialist unique to the repo.

Personal-tier Copilot has no org level — keep copies, and make the quarterly REFINEMENT pass diff them. Skip this layer below three repos.

Layer 3 — the workspace repo

Create it when a task description regularly contains more than one repo name. Layout:

```
workspace/                                ← its own git repo; NO CI; deploys nothing
├── <ws>.code-workspace                    ← folders: ".", "web", "mobile", "services/orders", …
├── workspace.json                         ← manifest: repos, contracts, delegation defaults
├── .gitignore                             ← every child clone directory
├── .github/
│   ├── copilot-instructions.md            ← workspace router (templates/workspace/copilot-instructions.md.template)
│   ├── agents/
│   │   ├── <ws>-orchestrator.agent.md     ← cross-repo coordinator; agents: [contract-guardian, service-mapper]
│   │   ├── contract-guardian.agent.md     ← REVIEW-ONLY: contract diff + consumer grep across folders
│   │   └── service-mapper.agent.md        ← read-only: who owns what; maintains SERVICE_MAP.md
│   ├── instructions/
│   │   └── cross-repo-contracts.instructions.md   ← applyTo: "**" — the contract protocol, always on
│   ├── skills/
│   │   └── delegate/SKILL.md              ← /delegate <repo> <handoff> — runs the child's orchestrator via the CLI
│   └── hooks/framework.json               ← optional, from templates/hooks/ (correction-capture only); not among the fourteen workspace
files
├── scripts/
│   ├── sync-repos.sh                      ← clone/fetch every repo in the manifest; never resets a branch
│   └── delegate.sh                        ← cd <child> && copilot -p … --agent=<repo>-orchestrator
├── docs/ai-context/
│   ├── SERVICE_MAP.md · CONTRACTS.md · HANDOFF_SCHEMA.md
├── web/  mobile/  services/orders/        ← gitignored clones
```

Plain clones (not submodules) driven by `workspace.json` + `sync-repos.sh`. Submodules pin SHAs, which fights active development. The `.code-workspace` file is what the teammate meant by "a workspace file" — it is the right artefact; it just should not be the only one.

No CI at the workspace level. Each repo deploys itself. The one optional job is a scheduled `contract-guardian` run that opens an issue on drift.

The workspace orchestrator's contract

A coordinator with no edit tools. Its job:

1. Restate the cross-repo task; read `SERVICE_MAP.md` and `CONTRACTS.md`; list the repos and the contracts between them.
2. **Order by contract** — producer first, additive only, deployed (check the producer's `PROJECT.md` §3, not its git log) before any consumer depends on it.
3. **One handoff per repo**, to that repo's `<repo>-orchestrator`, using the standard schema plus `repo:` and `contract_impact:`. Sequential for repos that share a contract.
4. Delegate with one of the mechanisms below; validate every `return:`; refuse to declare done while any `contract_impact.consumers_to_update` lacks a completed consumer handoff.

Three delegation mechanisms — pick by surface and by whether the child is written to

Mechanism A — the child's own Copilot CLI session (default for writes, and the only one that works from a script). `scripts/delegate.sh <repo> <handoff>` runs, *from inside the child directory* (the CLI has no `--cwd`; it loads the working directory's customizations):

```
cd "<repo path>" && copilot -p "$(cat handoff.yaml)" --agent="<repo>-orchestrator" \
  --allow-tool="<TOOL_SPEC>" … -s
```

The child's instructions, agents, skills and `.github/hooks/*.json` all load — its build-gate and doc-gate fire. Only the child's agents exist in that process, so **no name collisions**. The `return:` block comes back in the text output (a documented structured-output flag is not available on the CLI as of the verification date — the script extracts the YAML block). Pre-approve the tools the child needs with `--allow-tool=`; never `--allow-all-tools` on a repo you don't fully trust — `-p` runs that repo's hooks without a prompt.

Mechanism B — in-editor subagents (interactive cross-repo work in VS Code). The workspace orchestrator's `agents:` lists the child orchestrators by name (`web-orchestrator`, `orders-orchestrator`, …); VS Code loads them from the child folders; `runSubagent` invokes them; with `chat.subagents.allowInvocationsFromSubagents` enabled each child orchestrator can in turn invoke its own specialists. Every child's `applyTo:` instructions and hooks apply because they are workspace folders. The cost is the collision hazard from the table above: **only orchestrators are invoked by name from the workspace level** (unique by construction), and if two children ship same-named specialists, the child orchestrators' own `agents:` lists cannot disambiguate them — prefer Mechanism A for writes in that case.

Mechanism C – the cloud agent, one task per repo. The workspace orchestrator (or a human) files one issue per repo, in contract order, each carrying the handoff block in the issue body and assigned to that repo's orchestrator agent. The cloud agent loads that repo's `.github/hooks/*.json` and agents. There is no cross-repo run; sequencing is the protocol's job.

Two new handoff fields (additive – `schema_version` stays 1)

Outbound `repo:` and `contract_impact: {level, contracts, consumers_to_update}`; inbound `contracts_changed: [{contract, change, backward_compatible, consumers_grepped}]`. Full definitions in `docs/04-HANDOFF-SCHEMA.md`. A specialist that finds itself needing a second repo returns `status: blocked` with `recommended_next_agent` naming that repo's orchestrator – it never reaches across, and on the cloud agent it physically cannot.

The cross-repo contract protocol (`.github/instructions/cross-repo-contracts.instructions.md`)

The multi-client parity rules from Chapter 11, across repository boundaries:

1. **Producer first, additive only, deployed before any consumer depends on it.** A breaking change is add → move every consumer → remove; never one handoff.
2. **Grep every consumer in the same turn** and list them in `consumers_grepped` – including the zero-hit ones. Clients pin shapes; a rename fails silently at runtime.
3. **Never ship a consumer that hard-depends on a producer change that might not be deployed.** Probe or degrade, and say so on screen when degraded.
4. **Same heading ⇒ same endpoint.** Before building a panel that mirrors another consumer's, open that consumer's code and find what it fetches; record the pairing in `CONTRACTS.md`.
5. **One brain.** Logic in the producer; consumers render.
6. **One session for a whole cross-repo change.** Plan first, then delegate in contract order.

`workspace.json` and the `.code-workspace`

The manifest (`templates/workspace/workspace.json.template`) lists each repo's `path`, `url`, `default_branch`, `orchestrator`, `build`, `test`, `owns`, plus every contract (producer, consumers, spec path, versioning rule) and the delegation defaults. The `.code-workspace` file (`templates/workspace/workspace.code-workspace.template`) lists the same paths as folders, with the workspace root first, and sets `chat.subagents.allowInvocationsFromSubagents` for Mechanism B.

Scheduled workspace routines – the contract guardian gets a clock

The layout above already names the one workspace job worth scheduling ("a scheduled `contract-guardian` run that opens an issue on drift"). This section specifies it – and its two siblings – as standing routines (Chapter 13), because agent-era defects concentrate at the seams between repos: a contract changed in the producer and never propagated is invisible to every per-repo session until something breaks.

- **contract-drift-daily** – re-derive each contract's consumer list from the child clones and diff reality against `CONTRACTS.md` + `SERVICE_MAP.md`; open a workspace PR (or per-repo issues) on divergence. This is `contract-guardian` run on a schedule instead of on demand – same charter, same REVIEW-ONLY posture, plus the routine output contract.
- **service-map-freshener** – re-verify `SERVICE_MAP.md` rows (routes, owners, deploy targets) against the clones; a stale row gets a dated PR, not silent tolerance (Chapter 11's freshness contract, applied cross-repo).
- **workspace janitor** – prune stale clones and re-sync from the manifest (`sync-repos.sh`), so delegation never runs against a repo state nobody chose.

Routine writes follow the same delegation rule as everything else in this chapter: a routine that must *change* a child repo delegates to that child's own orchestrator session so the child's enforcement fires; the workspace-level routine itself stays read-only plus reports. Conventions, budgets and the catalog: `docs/13-STANDING-ROUTINES.md`.

What NOT to do

- **Don't put the specialists in the workspace repo.** They belong with the code they edit, or at the org level.
- **Don't invoke a child's *specialist* by name from the workspace session.** Names collide across folders. Invoke the child's orchestrator and let it route.
- **Don't expect one cloud-agent task to touch two repos.** It cannot. One task per repo, in contract order.
- **Don't use git submodules for the children.** Manifest + gitignored clones.
- **Don't give the workspace a deploy pipeline.**
- **Don't run `copilot -p` with `--allow-all-tools` against a repo you don't fully trust** – it runs that repo's hooks unprompted.
- **Don't fan out writes to a producer and its consumer in parallel.**
- **Don't carry a production approval across repos.** Per repo, per incident, current turn.

POC recipe (one afternoon)

1. Pick a service and one consumer that share a contract. Confirm each has a layer-1 install with a working `<repo>-orchestrator` (open the repo in VS Code, select the orchestrator, give it a single-repo bug). If not, do that first.
2. Create the workspace repo from `templates/workspace/`. Fill `workspace.json` and the `.code-workspace` with the two repos and the one contract. Run `scripts/sync-repos.sh`. Open the `.code-workspace` in VS Code.

3. Select `<ws>-orchestrator` and ask for something *additive* that crosses the contract ("add `estimated_delivery` to the order response and show it on the web order page").
4. Verify, in order: it read `CONTRACTS.md` and ordered producer → consumer; the producer handoff carried `repo:` and `contract_impact.level: additive`; the producer's own instruction files and hooks fired (Mechanism A: in the CLI output; Mechanism B: in the child folder's hook behaviour); the consumer return carried `consumers_grepped`; `git status` in the workspace root is clean.
5. Break it on purpose: ask for a *rename*. The correct outcome is a refusal until the task is re-issued as add-then-remove with both consumers listed.
6. Only then decide on layer 2: count identical agent files across the two repos.

Cross-links

- `docs/02-ARCHITECTURE.md` § Variations – points here for multiple repositories.
- `docs/04-HANDOFF-SCHEMA.md` – the base schema and the two cross-repo fields.
- `docs/11-PROJECT-TRUTH-AND-LEARNINGS.md` § Multi-client parity – origin of the contract protocol.
- `docs/10-MECHANICAL-ENFORCEMENT.md` – the hooks each child brings with it.
- `templates/workspace/` – every file in the layout above.
- Official (verified 2026-08-22): *Custom agents configuration*, *VS Code → Custom agents*, *VS Code → Subagents*, *Agent customization overview* (per-folder discovery), *About Copilot coding agent* (single-repo), *Copilot CLI programmatic reference*, *Hooks reference*.
- `docs/13-STANDING-ROUTINES.md` – the routine conventions the scheduled layer above relies on.

13 — Standing Routines (scheduled autonomy)

The question this chapter answers

Everything before this chapter runs when a person asks. The orchestrator decomposes a task (Chapter 2), hooks enforce discipline while a session runs (Chapter 10), and the truth files brief the next session (Chapter 11) — but nothing happens between sessions. This chapter adds the third leg: **routines** — narrow agent jobs that run on a schedule, produce small reviewable pull requests, and get *tuned* instead of babysat.

The pattern is not speculative. The Claude Code team at Anthropic ran it publicly on their own repos in Aug 2026: a fleet of daily routines (crash fuzzing, duplicate-abstraction unification, dead-code removal, flaky-test root-causing) opened **388 PRs in a few weeks, 180 of which were merged** after automated review plus human review, at roughly 1-in-50 noise. Nothing in the pattern is vendor-specific — Copilot's cloud agent and headless CLI are exactly the surfaces it needs. Everything below restates the practice stack- and domain-agnostically, folded into this framework's existing conventions.

Where routines sit

Leg	Runs when	Chapter
Interactive orchestration	a person asks	02, 03, 06
Mechanical enforcement	a session does something	10
Standing routines	**the clock says so**	**this one**

A routine is *not* a cron script with an LLM inside. It is a specialist (Chapter 3) with a written charter, an output contract, a review gate, and a tuning log — the same discipline the framework applies to interactive agents, applied to unattended ones. If a job needs no judgment (bump a version, rotate a log), use plain Actions automation; a routine earns its cost only where the work needs reading, reasoning, or writing code.

The seven conventions

1. One routine, one narrow charter. "Maintain the codebase" is not a charter. "Find statically unreachable code and remove it" is. A routine that needs the word "and" in its charter is usually two routines. The charter lives in a checked-in file (`templates/routine.md.template`) so it can be diffed, reviewed, and tuned like any other framework file.

2. Output is small, self-contained PRs — never direct pushes. A routine's writes reach the integration branch only through a PR (Pitfall 13 applies doubly to unattended writers — and the cloud agent's own PR-only workflow already enforces this shape). Small and self-contained is what makes ten routine PRs a five-minute review instead of an afternoon.

3. Every fix-PR carries a repro and a truth table. "No evidence, no claim" (Principle 3) does not relax when nobody is watching — it tightens. A crash fix shows the reproduction; a behavior change shows the before/after truth table, verified end-to-end on the real system, not a mock. A routine PR without evidence is closed without debate, and that closure feeds convention 6.

4. All routine activity reports to one place. One issue thread, one channel, or one dated log file per fleet — not scattered notifications. The point is that a person can read one surface each morning and know what ran, what it produced, and what died. Routines that fail silently are worse than no routines (see Pitfall 30).

5. A routine never merges its own PR. The gate is: automated review first (Copilot code review on every routine PR; a stronger review pass for anything touching money, auth, data deletion, or another repo's contract), then a human merge. The human's cost stays low *because* of conventions 2 and 3.

6. Wrong output tunes the routine, not just the output. When a routine PR is wrong, fix the PR if it's worth fixing — but always patch the routine's charter/prompt so tomorrow's run doesn't repeat it, and date the change in the charter's tuning log. This is the correction-capture loop (Chapter 10) applied to unattended work. Expect a real noise rate — the public reference fleet ran at ~1 in 50 — and budget it; a routine whose noise stays above its budget after two tuning passes gets retired, not tolerated.

7. Attempt caps and a verified retire path. Every routine carries a per-run budget (time or premium-request spend), an attempt cap after which it parks instead of grinding, and — critically — a *checked* completion write. One production team's generator pipeline ran for 17 days without a single job completing: the "done" write violated a database constraint, the error return was never read, and a reaper kept resurrecting the same satisfied jobs forever. The lesson generalizes to any unattended loop: **a best-effort write whose failure changes control flow must have its error read**, and "the routine ran" must never be reported as "the routine worked" (Pitfall 14, unattended edition).

Model and effort tiering

Match the model to the routine, not the fleet. The public reference fleet ran mostly on a mid-size frontier model, reserving the largest for a few hard routines — and noted that a smaller model is workable if you "spend a bit more time auditing PRs and iterating on routines' prompts, then adding in checks and guardrails." That trade is general: **cheaper model ⇒ tighter output contract, more automated checks, more sampling in review.** Where Copilot exposes model choice (cloud agent, CLI), set it per routine, and re-tier quarterly as models change (Pitfall 20).

A starter catalog

Charters that have earned their keep, restated generically — pick two, not ten:

- **Dead-code remover** — remove statically unreachable code. For *suspected* (dynamically reachable?) dead code: add logging first, check the logs next run, remove only then. The two-step is the difference between a janitor and a hazard.
- **Duplicate-abstraction unifier** — find similar-yet-slightly-divergent helpers/components and unify them behind one, with a truth table showing call-site behavior is unchanged.
- **Flaky-test root-causer** — pick the week's flakiest test, find the *root cause* (order dependence, time, shared state), fix it or delete it with a written justification. Never auto-retry as the fix.
- **Doc-drift checker** — diff Tier-1/Tier-2 docs (Chapter 7) against reality: dead links, stale counts, commands that no longer exist. Companion to the librarian (Pitfall 16), running daily instead of quarterly.
- **Crash fuzzer** (repos that ship an app) — drive the real app, find crashes, root-cause, open a fix PR with the repro attached.
- **Contract-drift checker** (workspaces) — see Chapter 12 § Scheduled workspace routines.
- **Workspace janitor** — prune stale worktrees, merged branches, dead clone directories (Pitfall 26's cleanup crew).
- **Hill-climber** — a routine wrapper around the hill-climb skill (`templates/skills/hill-climb/SKILL.md.template`): one metric, one target, one bounded iteration per run.

Running them on Copilot

Three mechanisms, in order of preference:

1. **Scheduled workflow → cloud agent (Mode 5).** A GitHub Actions `schedule:` workflow creates (or re-opens) the routine's tracking issue and assigns it to Copilot; the cloud agent works in its own environment and opens the PR. This is the most native fit — the cloud agent's PR-per-task shape *is* convention 2. Keep the charter file as the issue body's single source.
2. **Scheduled workflow → headless CLI (Mode 6).** `copilot -p` with `--agent=<routine-agent>` inside a cron workflow — the portable floor when you need a custom toolchain in the runner image. Note: whether `.github/hooks/*.json` fire in headless CI sessions is **not re-verified** — test it in your install and stamp a verified-on date before relying on hook enforcement inside routine runs; until then, treat the PR gate (convention 5) as the enforcement layer.
3. **Org-level automation** — where an organization schedules agents centrally, the same charter and gates apply; the mechanism changes, the conventions do not.

Give every mechanism a kill switch a human can flip without a deploy (disable the workflow, close the tracking issue with a label, an environment variable) — you will want it during an incident.

What NOT to do

- Don't let a routine merge, push to the integration branch, or touch production state directly — PRs only, gates always.
- Don't launch a fleet before the first routine has run for two weeks and had its noise tuned down — routines compound, and so does their noise.
- Don't give a routine agent a broad toolset "to be safe" — charter-narrow `tools:` , same as any specialist (Principle 5); an unattended agent with wide write access is your largest blast radius.
- Don't run a routine without a budget and an attempt cap — a runaway loop discovered on the monthly premium-request bill is the canonical failure (see also REFINEMENT check 12).
- Don't report "ran" as "worked" — completion means the *verified* completion write landed.
- Don't skip the schedule and run "when someone remembers" — that is not a routine, that is Chapter 6.

Cross-links

- `templates/routine.md.template` — the charter file every routine checks in.
- `templates/skills/hill-climb/SKILL.md.template` — the metric-loop skill routines can wrap.
- `docs/06-INVOCATION-MODES.md` § Scheduled runs — where routines sit among the modes (Mode 5/6 on a clock).
- `docs/10-MECHANICAL-ENFORCEMENT.md` — the correction-capture pattern is the tuning loop's interactive twin.
- `docs/11-PROJECT-TRUTH-AND-LEARNINGS.md` — routine tuning logs follow the LEARNINGS entry shape.
- `docs/12-MULTI-REPO-WORKSPACES.md` § Scheduled workspace routines — the contract guardian on a clock.
- `docs/08-COMMON-PITFALLS.md` — Pitfalls 29 (context weight) and 30 (unattended jobs need a verified retire path).